Secure Presence Verification System

Current validation and publication are recorded in current status and the 27 October synchronization checkpoint. Delivery PDFs are regenerated for that checkpoint. The 24 October ecosystem audit remains a historical record of the earlier local/remote gap.

Title page

The current attendance protocol adds mandatory rotating QR and acoustic proof to the identity/location checks. See protocol details, Mac runtime integration, and C003 evidence scope. Earlier physical results do not establish the new acoustic channel's classroom performance.

A two-layer attendance verification platform for educational institutions

College / University: **[Enter institution name]**

Department: **[Enter department]**

Submitted by: **[Enter student name(s)]**

Roll number(s): **[Enter roll number(s)]**

Project supervisor: **[Enter supervisor name and designation]**

Academic year / semester: **[Enter academic year and semester]**

Project abbreviation: **SPVS**. Source repository: Secure-Presence-Verification-System-SVPS---Attendance-Software. The inherited repository name uses SVPS; the implemented product and API use SPVS consistently.

Abstract

SPVS is an integrated student attendance system that combines biometric identity evidence with physical-presence evidence. The backend validates a randomized temporal face challenge, a real pretrained face embedding, a device-held signing credential, current GPS measurements and available Wi-Fi observations. It produces explicit present, rejected, review, retry or expired decisions. PostgreSQL transactions and unique constraints prevent duplicate successful attendance, while faculty corrections require attributable reasons.

The implementation uses FastAPI, SQLAlchemy, PostgreSQL, OpenCV SFace/YuNet, MediaPipe, a React institutional dashboard and an Expo student application. It includes actual API integration, migrations, automated tests, model checksums, dependency locks and submission artifacts. Verification evidence is separated from claims that require a physical phone or external attestation configuration. The system demonstrates layered resistance and accountable review without claiming to prevent every spoofing or collusion attack.

1. Introduction

Attendance is both an academic record and a security-sensitive institutional workflow. A presence claim affects eligibility, evaluation and student disputes. A simple token or register makes recording convenient but may provide little evidence about who submitted the claim or where that person was located. At the same time, a complex verification system can harm usability if it silently rejects students or overstates the reliability of noisy sensor data.

SPVS approaches the problem through two primary layers. Identity verification evaluates a consented face template, a fresh challenge and an authorized device. Physical presence verification evaluates location quality, the classroom geofence and supported Wi-Fi signals. Supporting authentication, academic configuration, reports and auditing make those layers usable in an actual college workflow.

2. Problem statement

Traditional methods allow shared tokens, proxy responses, remote submissions and unaccountable edits. A mobile client is an untrusted input source; it must not declare its own face similarity or distance from a classroom. Shared tokens, expired sessions and repeated requests require server-side controls.

Software-only verification cannot absolutely prove presence against a fully compromised phone or coordinated attackers. The engineering problem is therefore to enforce meaningful gates, expose uncertainty, preserve accountability and evaluate the limitations honestly. SPVS excludes surveillance cameras and additional beacon hardware, keeping the project reproducible with normal student phones.

3. Objectives

1. Implement a complete role-based academic attendance workflow.

2. Use actual model-generated face embeddings with consent and protected template storage.

3. Validate randomized temporal challenges on the server.

4. Bind evidence to an authorized device credential and a fresh session.

5. Calculate geofence and signal decisions from submitted measurements on the server.

6. Make retries, replay, concurrency and corrections deterministic and testable.

7. Provide understandable student outcomes and an accountable faculty review process.

8. Deliver source, documentation, measured test evidence and reproducible setup.

4. Existing systems and limitations

Manual registers are simple but allow proxy marks and require transcription. Static QR or code-based systems are convenient but shared tokens can travel beyond the classroom. Pure GPS systems are affected by indoor uncertainty and spoofing. Face-only systems may accept photograph/video presentation if identity matching is confused with liveness. Device identifiers can be copied and do not establish cryptographic possession.

Adding more signals does not automatically establish strong assurance. A high weighted score can conceal a failed security gate if the decision algorithm is poorly designed. SPVS instead enforces mandatory identity/session/device conditions before evaluating supporting location evidence, then routes ambiguity to review or retry.

5. Proposed system

The student receives an institution-managed account, registers one device and consents to biometric enrollment. Faculty open a session for a course and configured classroom. The student requests a bound challenge, performs a short face sequence and collects real foreground location and supported Wi-Fi data. The app signs exact evidence bytes and submits them to FastAPI.

The server checks access, enrollment, session timing, device signature and challenge ownership. It detects and aligns the face, calculates SFace embeddings, compares each frame and analyzes temporal landmarks. Only after these gates pass does it evaluate geofence uncertainty, freshness, Wi-Fi policy and anomalies. A successful decision inserts one attendance record; other states remain attempts that can be disputed and reviewed.

6. Related work and implementation references

The system builds on documented OpenCV face detection/recognition APIs, MediaPipe facial landmarks, modern password hashing and relational transaction controls. OpenCV's published face tutorial supplies an example matching baseline, not an experimental result from this project. Android documentation explains why Wi-Fi scanning and location permissions cannot be assumed to work uniformly. Google and Apple attestation guides describe provider verification boundaries that require external configuration.

The bibliography lists official documentation, pinned model source and actual test-image provenance. No invented publication, benchmark accuracy, institutional deployment or user study is claimed.

7. System requirements

The backend requires Python 3.11/3.12-compatible dependencies, PostgreSQL 15+ (16 in Compose), the pinned model files and generated JWT/Fernet secrets. The dashboard requires a supported Node runtime and a modern browser. Student verification requires a camera-equipped Android/iOS phone, foreground location permission and a reachable API. Android native testing needs a development build, compatible JDK/SDK and the included local Wi-Fi module. iOS requires macOS/Xcode signing and exposes no general Wi-Fi scan API in this implementation.

The primary developer target is a MacBook Air M4. Native ARM64 Python/Node and Docker Desktop are documented. Automated execution in this submission occurred on Linux x86_64; that does not establish Apple Silicon runtime testing.

8. Architecture

The architecture has client, API/service and persistence tiers. The dashboard performs staff operations through a same-origin API proxy. The student app holds credentials in OS secure storage and sends signed evidence. FastAPI enforces roles and scopes, coordinates verification services and records outcomes in PostgreSQL.

The architectural, sequence and decision diagrams in `SYSTEM_ARCHITECTURE.md` are part of this report's generated PDF appendices. OpenCV and MediaPipe inference are synchronous; FastAPI synchronous handlers use its worker thread

pool. A per-worker lock guards model instances. Database rate limits and transactional locks remain shared across API workers.

9. Database design

Twenty normalized application tables cover accounts/subtypes, courses/enrollment, classrooms/Wi-Fi, policy, devices/resets, templates, sessions/challenges, attempts/records, events/corrections/disputes, refresh tokens and rate buckets. Foreign keys preserve ownership and attribution. Unique student/session records prevent duplicate official attendance. A partial unique active-device index enforces the normal one-device limit.

Attempts retain reasons and a coarse evidence summary rather than raw images. Corrections retain the actor, reason and previous/new status. Refresh tokens are hashed and grouped into revocable families. The explicit Alembic migration and ER diagram are included in the database appendix.

10. Implementation methodology

Development proceeded from relational/authentication foundations to academic workflows, identity/presence services, client integration, tests and documentation. Each API operation uses a Pydantic input contract. Unknown fields, invalid coordinates and malformed timestamps are rejected. Role-scoped queries and object checks protect backend data independently of UI navigation.

The dashboard provides labeled forms, searchable tables, status labels, real aggregate charts and scoped exports. The mobile app provides enrollment consent, permission handling, sequential camera capture, location acquisition, signing and result/history screens. Android reads recent real scan results; iOS reads only permitted current-network BSSID. Both report unavailable evidence explicitly.

11. Identity verification design

YuNet detects exactly one usable face; SFace generates a normalized 128-dimensional embedding from the aligned crop. The enrolled template is encrypted under a key separate from JWT signing. Every frame must match the reference at the configured threshold. Enrollment frames also match one another before averaging.

MediaPipe supplies facial landmarks. A blink requires open/closed/open eye-aspect transitions. Head actions use normalized nose displacement and stable consecutive observations. Frames must be ordered, within the challenge period and separated by plausible time intervals. Challenge IDs are unpredictable and single-use. These controls provide basic replay resistance, not certified capture provenance or comprehensive deepfake resistance.

12. Physical presence design

Haversine distance is calculated on the server. Location age, future timestamps, uncertainty and reported mock indicators are checked before accepting a geofence. An uncertainty circle fully outside the radius rejects; an overlapping boundary routes to review. Android Wi-Fi observations are matched by known BSSID and minimum RSSI, requiring at least half of configured references when available.

An explicit Android policy controls missing Wi-Fi: review, reject or reduced-assurance GPS-only. The separate iOS policy defaults to review, with explicit retry/GPS-only options. A coarse travel rule flags implausible movement over short periods. Unconfigured platform attestation never returns a verified result. Numerical risk communicates concern and never bypasses a failed mandatory gate.

13. Security design

Passwords use Argon2id. JWTs validate issuer, audience, expiry and token type. Refresh sessions rotate transactionally and revoke their family after reuse. Device signatures cover both an idempotency key and exact JSON bytes, using a native P-256 private key and registered server-side public key (legacy HMAC remains readable for migration).

Student/challenge/session locks protect the attendance transaction. A retry returns the existing attempt only when the signed evidence matches. A changed payload conflicts, and a consumed challenge cannot be reused with a new key. A faculty correction uses the same session lock and produces a separate audit trail. Input body/decoded-image bounds and shared rate buckets constrain abuse.

14. Testing and validation

The project includes unit, database integration, client interaction, browser and real-model smoke tests. PostgreSQL-backed tests run the migration and verify role restrictions, CRUD, refresh replay, device mismatch/revocation, consent, session expiry, duplicates and concurrent retries. Rule tests cover location and temporal sequences. Browser tests interact with the actual running API and database. Mobile logic tests check signing interoperability and decision wording; Metro/Hermes export checks bundle construction.

The API happy-path tests explicitly replace vision to isolate transaction behavior. Real-model tests separately compare public portraits and reject zero/multiple faces. Neither proves full live-phone verification. Current commands, counts and outcomes are included from `TEST_RESULTS.md` in the generated report. Failed intermediate runs were investigated and are not represented as passing results.

15. Actual results

The source contains functional backend, dashboard and student app implementations rather than generated placeholders. Authentication, academic configuration, sessions, reports and audits have been exercised against PostgreSQL. Real pretrained face-model inference has been executed. Verification state handling is connected to the API and constrained database writes. The dashboard builds and browser workflows connect to actual endpoints; mobile logic and bundle checks are recorded separately.

The final result table is generated from executed test result files. No population biometric accuracy, real-device liveness success rate, production latency, enrollment count or deployment adoption is reported because those studies were not performed. Native build status, remaining upstream dependency advisories and provider limitations appear in the verification appendix.

16. Limitations

The system cannot guarantee presence against compromised clients or coordinated credential sharing. Basic landmark liveness can be attacked, GPS has indoor uncertainty, Wi-Fi is platform-dependent and attestation is unconfigured. Thresholds need institution-specific calibration and a consented evaluation dataset. Access tokens have an expiry window after logout; audit storage is not externally immutable. Large-class performance, retention automation, key rotation and fully managed production deployment require further engineering.

The current Expo dependency tree has unresolved upstream advisories recorded in the limitations appendix. These must remain visible in a college evaluation and future release decision. Real-device and Apple Silicon checks remain separate from cloud test results.

17. Future enhancements

Priorities are provider-backed integrity validation, stronger capture provenance/presentation-attack detection, accessible alternative attendance workflows, calibrated thresholds, measured false-accept/false-reject behavior, pagination, operational retention and protected external audit export. Future work should be based on observed weaknesses and institutional requirements. Excluded beacon, ultrasound, surveillance and blockchain hardware/systems are not proposed as hidden dependencies of this implementation.

18. Conclusion

SPVS demonstrates how identity and location evidence can be combined with server-side enforcement and attributable review in an actual attendance application. The strongest implemented properties are explicit decision gates, relational consistency, evidence-bound retries and accountable corrections. Its practical value depends on honest handling of uncertainty and measured device behavior rather than an absolute anti-proxy claim.

19. References

The full bibliography is in `REFERENCES.md` and is included in the report appendix. It contains official framework, model, operating-system and security documentation plus the actual public test-image sources and relevant unresolved advisories.

20. Appendices

The generated PDF appends architecture/ER/sequence diagrams, the API map, biometric and security design, threat model, current test results, requirements traceability and known limitations. Editable source Markdown and the packaging script are included in the ZIP so that the student can add genuine institutional measurements and academic details without rebuilding the software from scratch.

Cross-platform completion update

This continuation adds native Android Keystore/iOS Secure Enclave P-256 signing, nonce-based device registration, Apple App Attest verification architecture, current-network-only iOS Wi-Fi evidence, precise-location handling and a separate audited iPhone presence policy. Shared enrollment/liveness/session screens continue to use the real backend model pipeline. Automated evidence distinguishes real-model smoke tests from synthetic API/certificate tests, native compilation from JS exports, and unperformed physical-device validation. The appended cross-platform chapter and final verification report are authoritative for release outcomes; no biometric accuracy or provider-backed success has been invented.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Current SPVS status

Checkpoint: **27 October 2026, Asia/Kolkata**. Publication: PR #7, building on the upstream QR/acoustic PR #6. The PR/Git refs are authoritative for current publication state.

Read runtime integration, C003 observations, current verification, and synchronization.

Area	Current state
Integration checkpoint	`0fc235f916fb662db71c298091255ec10f0917f6`, based on upstream main `404eed8`
New attendance policy	Mandatory rotating QR + acoustic proof, alongside existing identity/device/GPS/platform-presence gates
Local database	Backed up privately; migrated to `9bd32a88f101`; existing counts and records preserved; no schema drift
Services	API/dashboard restarted; health/readiness and new presence route verified
Local automated tests	146 backend + 64 mobile + 11 dashboard + 10 native Wi-Fi = 231 passed
Separate upstream evidence	3 browser tests passed in cloud; not rerun against the live local demo database
Native builds	New Android arm64 APK, iOS simulator and signed-device app built with SpvsAcoustic; signatures/configuration checked
Device installation / new physical tests	Not performed; wait for explicit user readiness in the classroom
C003 configuration	Separate classroom, 50 m radius, two provisional 5 GHz references at -70 dBm; one of two matches suffices under existing rule
Earlier C003 inside trial	Automatic PRESENT / VERIFICATION; Wi-Fi ratio 0.5; pre-v2 flow
Later distance trial	User-confirmed hostel attempt rejected OUTSIDE_GEOFENCE at reported 151.1 m; Wi-Fi was not evaluated
Near-door validation	Calibration scan collected with door closed; fresh Wi-Fi-specific attendance comparison not completed
Earlier Android demo	Registration/enrollment, pending submission, manual CORRECTION, resolved dispute and phone history observed
iOS policy	Local review retained; dynamic requires a separate audited choice; App Attest remains unconfigured
Fresh npm audit	4 high mobile affected entries, zero dashboard; backend vulnerability scan historical
Production	Real QR/audio interoperability, leakage/relay, calibration and operational release gates remain open

Earlier dated reports describe their original checkpoints. Neither the old automatic C003 result nor native compilation proves physical QR/audio success. The old installed apps must be updated before the next trial. No physical test, session challenge or audible playback was started during this synchronization.

Printable records: runtime PDF, C003 PDF, sync PDF. See test results, physical plan, and release inventory.

Git publication excludes private backup/database data, credentials, signing keys, raw network identifiers, exact coordinates, templates, frames/audio and raw UI captures. It does not deploy a live cloud service or replicate the local database.

Cross-platform implementation

This completion extends commit `b19777df3cb3210330075c702e5e98f487213a5d`; FastAPI/PostgreSQL, React dashboard and Expo/React Native remain the architecture. The source supports Android and iOS. See current status and current verification for observed builds/tests. The historical cloud record is HISTORICAL_CLOUD_COMPLETION_2026-10-23.md. Source implementation is not physical-device validation.

Shared application and platform adapters

The physical Android continuation now verifies the basic Motorola workflow through authenticated demo faculty API correction and phone history. The latest shared sign-in change uses an email keyboard/autofill hint, rejects embedded whitespace and maps server field errors to actionable messages. No native ABI or backend schema change was required.

Authentication, enrollment and deletion/re-enrollment, session selection, randomized camera capture, result, history and API recovery use the same screens. `src/platform` owns native device keys, OS selection, Wi-Fi APIs and simulator API defaults. `capture.ts` executes the server's action order and expiration, cancels on backgrounding, and records capture-start timestamps. It does not calculate a successful biometric verdict on the client. `signals.ts` requests foreground location, checks precise accuracy where available and submits the actual fix time and horizontal uncertainty. Failed requests display recovery instructions. Camera temporary files are deleted in finally blocks.

Signal	Android	iOS	Server treatment
Face images	Expo Camera front camera	Expo Camera front camera	Same YuNet/SFace and MediaPipe inference
Liveness	JPEG still sequence, randomized prompts	Same protocol	Sequence, timing, duplicate-frame and identity gates
Device signature	Android Keystore P-256	Secure Enclave P-256 on physical devices; Keychain on simulator	ECDSA SHA-256, DER signature, registered SPKI key
Nearby Wi-Fi	Recent OS readings or awaited scan-completion callback, measured BSSID/RSSI	Not available to ordinary apps	Never synthesized
Current Wi-Fi	Scan evidence	NEHotspotNetwork.fetchCurrent, BSSID only	Separate current-network signal; no invented RSSI
Location	Expo location provider; coarse/full permission	Core Location through Expo and native accuracyAuthorization	Freshness, uncertainty, geofence, mock flag, travel checks
App attestation	Play Integrity remains unconfigured	App Attest native API plus server verifier	Unverified evidence is never reported as verified

Permissions and reproducible projects

`app.config.js` and `plugins/withSpvs.js` configure bundle/package `edu.spvs.student`, display name SPVS Student, portrait orientation, icons, launch screen, iOS 16.4 minimum, camera/location/local-network descriptions and Android backup exclusion. Access Wi-Fi Information is disabled by default for Personal Team compatibility. Supported Apple Developer Team builds opt in with `SPVS_IOS_WIFI_INFO=1` before prebuild and require a matching provisioning profile. Unsetting it or setting `0` removes only the Wi-Fi entitlement, including one left by a previous build; the native current-network implementation remains. Set the same variable when running `scripts/verify_native_config.py`. App Attest is separately controlled by `SPVS_APP_ATTEST_ENV=development` or `production`; its existing behavior is unchanged. No background location, background microphone or unrestricted iOS Wi-Fi scan is requested. Foreground microphone permission is now required for bounded QR/audio attendance capture; raw PCM is not uploaded or stored. Android location permission gates scan access; scan throttling and OEM restrictions can return unavailable.

Run `npm ci` then `npx expo prebuild --no-install --no-clean`. Expo 58 defaults to clearing native directories unless `--no-clean` is supplied. Generated Android and Xcode projects are derived artifacts; only the resolved iOS Podfile.lock is retained in Git. Local Swift/Kotlin Expo modules and podspecs are included. `python3 scripts/verify_native_config.py` checks generated files, not native compilation. The CocoaPods Gemfile pins 1.16.2; run `bundle install` before `bundle exec pod install`. The local Mac validation records the genuine CocoaPods resolution and Xcode run; the original Linux report remains historical.

Release transport requires an HTTPS API endpoint. For an isolated local development network only, set `SPVS_DEV_HTTP=1` before prebuild and use a development client. Production configuration defaults to TLS and disables Android cleartext. Do not use local HTTP for real student data. Android emulator defaults to 10.0.2.2; iOS simulator defaults to localhost. Set EXPO_PUBLIC_API_URL explicitly for physical phones and release builds.

Device binding and lifecycle

Registration requests a short-lived server nonce. The client signs `SPVS-REGISTER-v1.<nonce>.<platform>.<SHA256(SPKI bytes)>` using its newly generated P-256 key; the server verifies password confirmation, ownership, expiry, signature and the single-active-device constraint. Evidence signs the exact UTF-8 bytes `<idempotency_key>.<payload>`; the signature is base64 DER ECDSA/SHA-256. Neither platform exports a private key to JavaScript. The iOS physical-device path does not silently downgrade from Secure Enclave when key generation fails. Simulator Keychain keys have lower assurance.

A storage label is a client report, not proof of hardware protection. The server exposes this distinction in its admin device list. Old HMAC registrations continue to work for migration; new shared-secret registrations are disabled by default (`ALLOW_LEGACY_DEVICE_REGISTRATION=false`). Test fixtures explicitly enable legacy compatibility. Retire old devices through the audited reset process.

Native installation IDs and stored key aliases detect reinstall/key loss on submission. Signing never silently creates a replacement key. A lost/revoked credential requires an administrator-approved reset. iOS Keychain items can survive uninstall; an installation ID mismatch prevents their silent reuse. The existing device remains authorized until reset approval. Each student's local metadata is stored separately, device-only, in SecureStore. Native storage compromise, rooted devices or injected clients are not solved by a client-reported storage label.

Apple App Attest setup and protocol

1. In the Apple Developer portal register the exact bundle ID and enable App Attest and Access Wi-Fi Information. Use a matching development/distribution provisioning profile. App Attest requires a supported physical device; simulators return unsupported.

2. Retrieve Apple's **App Attestation Root CA** from Apple's official certificate authority page over verified HTTPS. Independently verify its SHA-256 fingerprint through a trusted institutional setup process. Do not accept a root supplied by

a client or a certificate from an arbitrary blog. Store the PEM outside the repository.

3. Configure the backend with `APPLE_APP_ID=<TeamID>.edu.spvs.student`, `APPLE_ATTEST_ENVIRONMENT=development` (or production), `APPLE_ATTEST_ROOT_PEM=/run/config/apple-app-attestation-root.pem` and `APPLE_ATTEST_ROOT_SHA256=<verified hex fingerprint>`. The certificate is public, but these are deployment trust settings. Mount the PEM read-only in Docker. All four values must be set; production/development separation is enforced.

4. Set the corresponding `SPVS_APP_ATTEST_ENV` before prebuild or use the EAS profile, sign and install the development build. Register the SPVS device, then choose Register Apple app attestation.

5. `/attestation/challenge` supplies a random one-use nonce bound to user and device. Native `attestKey` hashes `SPVS-ATTEST-v1.<nonce>.<device_id>.<key_id>`. The server decodes bounded CBOR, verifies the pinned certificate chain, validity, basic/key-usage constraints, nonce extension, app-ID hash, environment AAGUID, zero initial counter, credential ID, public-key hash and COSE key consistency. It stores the public key only on success; failed registration consumes its challenge.

6. Each attendance request generates an assertion over SHA256 of the exact signed request bytes. The backend checks the app hash and ECDSA signature and atomically advances a strictly increasing counter under a database lock. Replays and out-of-order assertions cannot pass. Exact authenticated attendance retries return the existing result without double counting.

The verifier is covered by synthetic certificate/signature tests. It has **not** been validated with a genuine Apple attestation object or physical iPhone. Apple receipt risk assessment and distribution-scale fraud monitoring are not implemented. Complete a provider interoperability review before enabling institutional attestation requirements. Apple's root and developer identity are intentionally not fabricated or bundled.

Play Integrity is a fail-closed boundary: no decoded client verdict is trusted. To finish its provider adapter, configure a Google Cloud project linked to Play Console, package identity and allowed signing-certificate digests, obtain a standard Integrity token bound to SHA256 of the exact request, decode it through Google's authenticated server API, enforce request package/hash/timestamp and app/device verdicts, and add replay/quota/error tests on Play-distributed physical devices. This release does not claim that integration is configured or verified.

Presence policy and assurance

The Android scanner reuses readings at most 15 seconds old, otherwise waits up to eight seconds for a successful system scan broadcast. Concurrent calls share one scan; timeout/cancellation removes listeners. Coarse and precise location permissions are requested together and precise access is required. Settings includes a local-only Wi-Fi setup check for reference collection. See the field procedure.

The current iOS `review` policy always requires review, including when the current BSSID matches. There is no separate automatic Wi-Fi-match approval mode. The Personal Team development build lacks the Wi-Fi entitlement, and the local demo classroom currently has no AP references. These are configuration and policy limits; the Android scan fix does not change them.

Android retains `wifi_unavailable=review|reject|gps_only`. iOS has a separate audited `ios_presence=review|retry|dynamic`, default review. The dynamic option requires server-verified QR/audio plus mandatory identity/device/liveness/GPS gates; it never overrides failed identity or a required unverified attestation. Every attempt records platform, policy, reported key storage and presence assurance. Missing Wi-Fi is distinct from a mismatched observed network. A permitted iOS current BSSID does not raise GPS to multi-AP fingerprint assurance. Reduced-accuracy location requires retry even when a client reports a small numeric radius. GPS coordinates and Wi-Fi observations remain potentially spoofable client evidence.

Camera protocol and limits

Both camera adapters return processed JPEGs at width 640, front camera with mirror disabled. Android YUV and iOS pixel-buffer formats do not cross the API; Expo handles sensor conversion and orientation before JPEG encoding. ImageManipulator resizes and recompresses, and the server validates JPEG dimensions, lighting, blur, exactly one face, landmarks and 128-dimensional normalized embeddings. The capture loop targets 170 ms intervals, allows a 350 ms prompt transition, and requests slow blink/open-close-open evidence. Actual capture speed and front-camera orientation are hardware dependent. Capture-start timestamps are adjusted by server clock offset; they are signed but not trusted hardware timestamps. Reject gaps outside server bounds and interrupted/expired captures; never forge timing to make a sequence pass.

Server inference and temporal tests do not establish live-camera usability, biometric accuracy, sophisticated presentation-attack resistance or injection resistance. Verify blink capture rate, left/right orientation, lens/mirroring, accessibility, reduced GPS, Wi-Fi throttling, airplane mode, key loss and network retry on real Android and iPhone hardware before a demonstration with students.

Native build procedures

See SETUP_AND_DEMO_GUIDE.md and `scripts/build_mobile_macos.sh`. Android builds require JDK 17, the SDK/NDK versions emitted by the pinned Expo prebuild and Gradle's trusted repositories. iOS requires macOS, a supported Xcode version (the pinned React Native helper declares at least 16.1; use a current supported Xcode), command-line tools, Ruby/Bundler and CocoaPods. Simulator compilation does not verify Secure Enclave or App Attest. Physical iPhone builds require signing and matching capabilities. TestFlight requires an Apple Developer membership, App Store Connect record, distribution provisioning and a genuine archive/upload. No unsigned file is renamed to IPA.

Policy v2 retires the iOS gps_only option. Current iOS choices are review, retry and dynamic; the migration maps an existing gps_only choice to dynamic while retaining stricter review/retry settings. Dynamic requires completed QR/audio proof alongside mandatory identity/device/GPS gates; a global requirement for attestation still applies. The local review policy was not changed. QR/audio does not independently attest a platform or sensor and can be relayed. See the protocol.

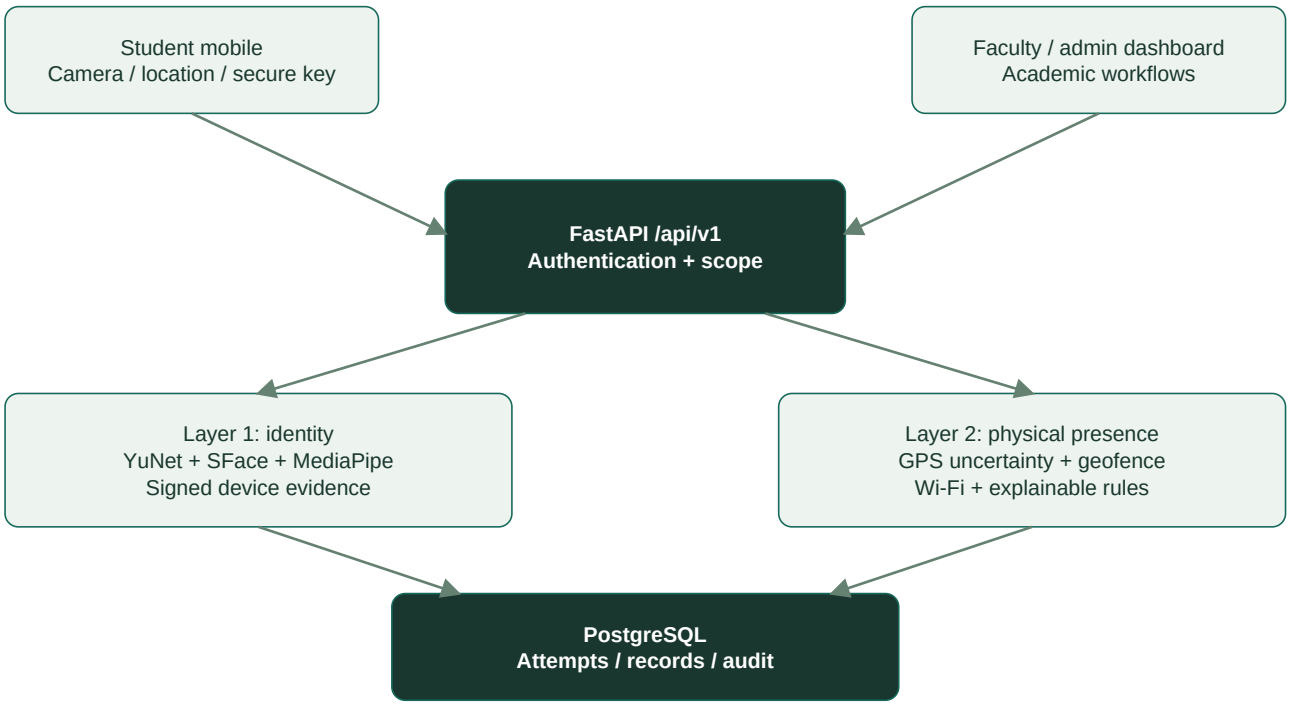
System architecture

Mandatory policy-v2 presence exchange

Current attendance adds a faculty QR display/playback channel, PresenceQR and PresenceProof state, and the student's signed QR/audio redemption before the existing face/location flow can automatically pass. The student UI performs QR/audio before face capture; server proof checks remain mandatory even for a client that skips the UI. See full protocol.

The diagrams below show the **core identity/GPS/Wi-Fi subflows**, not the complete policy-v2 exchange. QR/audio redemption occurs after challenge issuance and before the UI's face capture; the server cannot turn the core subflow's provisional PRESENT into automatic attendance without the required dynamic proof.

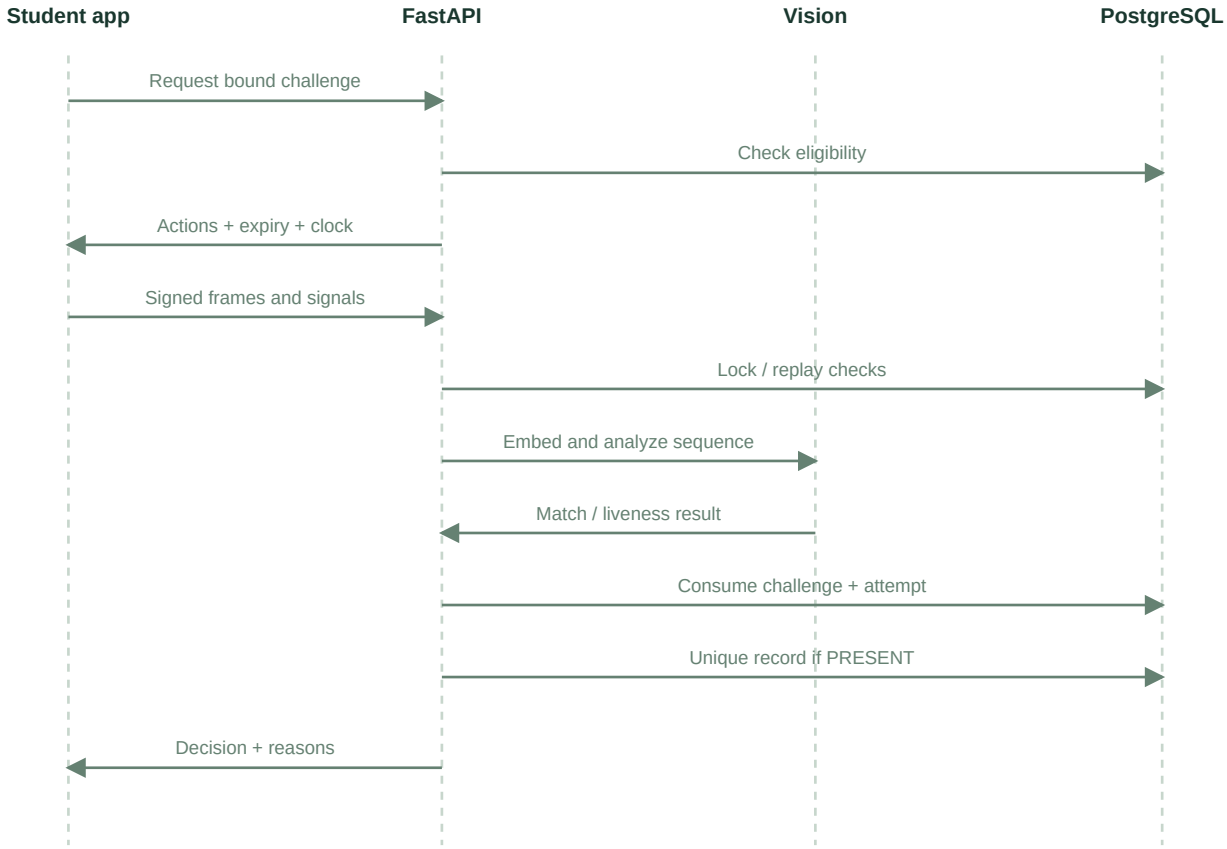
Component architecture



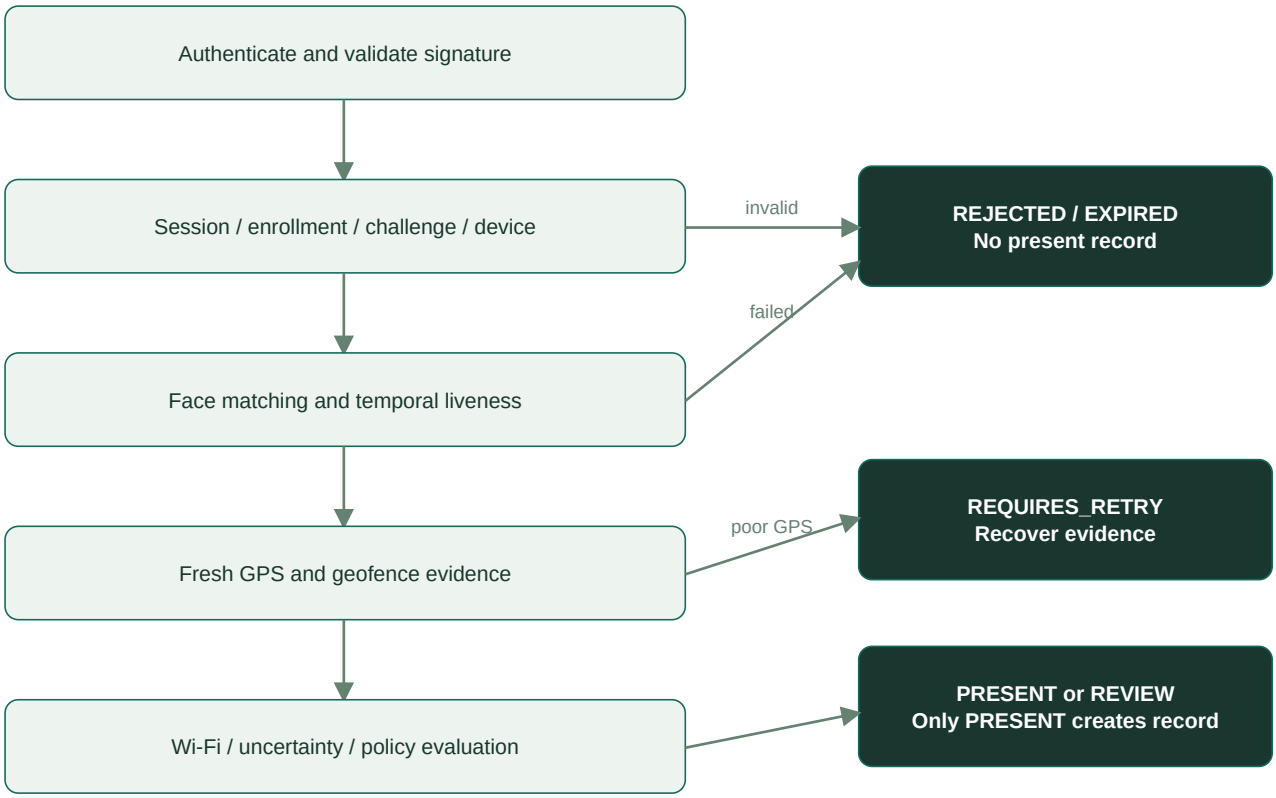
The backend uses synchronous SQLAlchemy sessions in FastAPI synchronous route handlers, which execute in the framework's thread pool. This is deliberate: OpenCV/MediaPipe inference is synchronous and guarded by a process-local model lock. The application does not block an async endpoint with a synchronous database call. Multiple API workers share PostgreSQL rate limits and locks; each worker has its own model memory. Capacity and fair scheduling require load testing before institutional deployment.

The dashboard uses a same-origin `/api` route through Vite in development or nginx in containers. Tokens remain in dashboard memory. The mobile app stores tokens and per-user key metadata in SecureStore; new private keys stay in native Keystore/Secure Enclave storage. Its camera cache files are deleted after conversion; no raw frames are saved by the backend.

Verification sequence



Decision flow



Authentication, malformed payload and signature failures are HTTP errors; signed, attributable verification outcomes are persisted as attempts. This distinction prevents unauthenticated requests from inventing someone else's attempt history.

Cross-platform completion update

Native device keys now live in `spvs-security` (Kotlin/Swift), with shared TypeScript adapters. App Attest registration uses dedicated one-use security challenges; assertions bind the exact signed request and advance a PostgreSQL counter. iOS current-network evidence is separate from Android scan measurements. See CROSS_PLATFORM_IMPLEMENTATION.md for trust boundaries and native-build limits.

Database design

Current QR/audio migration

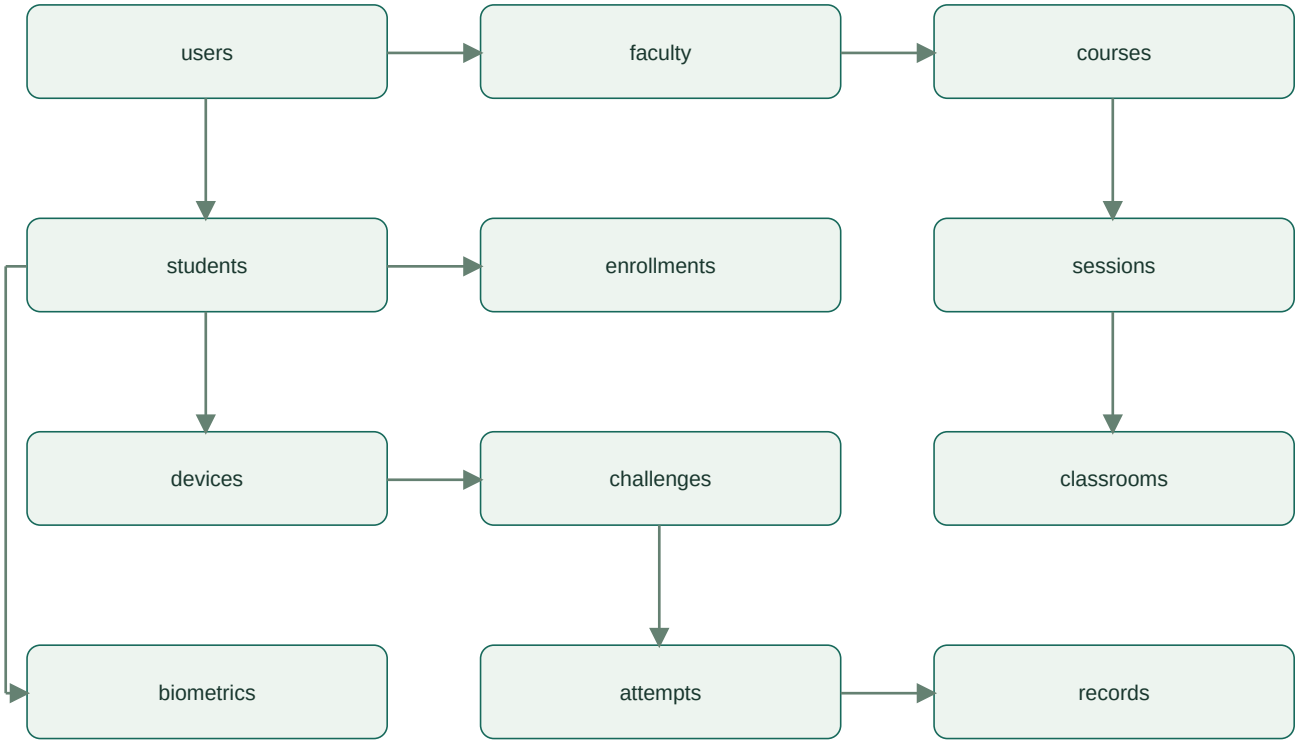
Migration `9bd32a88f101` adds presence_enabled/presence_epoch to attendance sessions, a presence_qr table for rotating session/classroom/epoch-bound tokens, and presence_proofs keyed by attempt challenge with QR verification, listening readiness, hashed audio response, deadlines, consumption and idempotency state. Unique QR/student redemption and row locks enforce the protocol. Existing attempt/record history is retained; no raw audio column is introduced.

The migration maps legacy iOS gps_only policy to dynamic while retaining review and retry. The local update used a private backup, preserved existing record counts and C003 configuration, and passed the model-drift check. The schema diagram below shows the earlier core entities; this paragraph and protocol describe the additional policy-v2 relationships.

PostgreSQL 16 is used in Compose. The initial migration creates 20 application tables plus Alembic's version table. IDs are application-generated UUID strings. Timestamps are timezone-aware. There are no raw-image columns.

Table	Purpose / important constraint
users	Unique email, password hash, current role and active status
students	User FK and unique roll number
faculty	User FK and department
courses	Unique code, assigned faculty FK
enrollments	Composite primary key: course and student
classrooms	Unique name, latitude, longitude, radius
wifi_references	Unique classroom/BSSID and minimum RSSI
policies	Singleton verification settings
devices	Native public-key metadata or encrypted legacy shared secret; one active device per student
device_resets	Requested reason, status and administrator reviewer
biometric_profiles	Student primary key, encrypted template, model version, consent timestamp
attendance_sessions	Course/classroom FKs, start/end and explicit closure
challenges	Student/device/session bindings, purpose, random actions, expiry and used flag
attendance_attempts	Unique student/idempotency key, payload hash, decision, reasons and evidence summary
attendance_records	Unique student/session, optional attempt FK, status and source
security_events	Actor/session, severity, structured details and review flag

Table	Purpose / important constraint
corrections	Record/actor FKs, previous/new status and mandatory reason
disputes	Student/attempt, reason, resolution and reviewer
refresh_tokens	Hashed random token, family, expiry, used and revoked flags
rate_buckets	Shared fixed-window counters keyed by account or connection source



Core foreign-key relationships; full schema and remaining tables listed in the database chapter.

Verification locks the student row, then challenge and session, before consuming the challenge and writing the attempt/record. The same session row serializes a manual correction against verification. Device reset approval locks the student before revocation. A refresh-token rotation locks its token row; reuse revokes the entire refresh family. These transactions complement, rather than replace, unique constraints.

Subtype rows are retained when a current role changes because historical foreign keys must remain valid. Authorization always reads the current user role. Deleting referenced academic entities is intentionally restricted; deactivation and enrollment removal are supported while history remains attributable.

Location summaries retain coordinates rounded to three decimal places for coarse travel anomaly checks, plus distance, accuracy and age. Define an institutional retention schedule before deployment. The current application does not implement automated retention purges or an immutable external audit sink.

Cross-platform completion update

Migrations `5c9d2a1e8b70` and `7a2d13ce4f90` add device key_algorithm/public_key/key_storage, policy ios_presence, security_challenges (user/device/purpose/nonce/expiry/used), app_attest_keys (globally unique key ID, unique device, public key, monotonic bigint counter/environment), and challenge enrollment_request_hash. Existing device rows default to legacy HMAC. The enrollment hash permits exact authenticated HTTP retries; different requests cannot replay a used challenge. Device and student locks serialize revocation/signature decisions. Raw camera frames are never stored in these tables.

API documentation

Policy-v2 classroom presence endpoints

The current schema includes staff-scoped `POST /sessions/{sid}/presence/control`, `GET /sessions/{sid}/presence/display`, and `POST /sessions/{sid}/presence/play/{attempt_id}`. Students submit signed `qr`, `ready`, `audio` or `failure` messages through `POST /presence/evidence`. Complete QR/audio proof is mandatory for new automatic attendance; declared capture failure produces a review attempt with identity NOT_COMPLETED.

Attendance challenges use a 120-second lifetime; enrollment retains its configured lifetime. Existing unconsumed attendance challenges may be reused, and the attempt budget counts issued challenges. Five-second QR rotation, at most ten-second QR validity, a server-clock five-second acoustic deadline, nonce binding, consumption and idempotency are described in the protocol. Use `docs/openapi.json` for field-level contracts. Enrollment and previous records are not retroactively changed by migration `9bd32a88f101`.

The authoritative machine-readable contract is `docs/openapi.json`, exported from the implemented application, and the live `/openapi.json`. The versioned API prefix is `/api/v1`. Protected operations use `Authorization: Bearer <access token>`.

Endpoint groups

Group	Main routes	Authorization
Authentication	POST `/auth/login`, `/auth/refresh`, `/auth/logout`; GET `/auth/me`	Public token exchange; authenticated profile
Registration	POST `/auth/register`	Disabled by default; student-only when enabled
Accounts	GET/POST `/users`; PATCH `/users/{id}`	Administrator
Academics	GET/POST `/courses`; GET/POST/DELETE enrollment routes	Course-scoped reads; administrator writes
Classrooms	GET/POST `/classrooms`; PUT `/classrooms/{id}`	Staff read; administrator write
Sessions	GET/POST `/sessions`; GET `/{id}`; POST `/{id}/close`	Enrolled student read; assigned faculty/admin manage
Biometrics	GET/DELETE `/biometrics`; POST `/biometrics/enroll`	Student's own template
Device	GET/POST `/devices`; reset request/list/approval routes	Student ownership; admin reset approval
Evidence	POST `/challenges`, `/attendance/verify`	Authenticated student and active bound device
Review	GET/POST `/disputes`; POST `/{id}/resolve`; GET/POST `/corrections`	Student ownership; scoped staff decisions
Audit	GET `/events`; POST `/{id}/review`	Scoped faculty or administrator
Records	GET `/attempts`, `/records`	Student own data or scoped staff data
Reporting	GET `/overview`, `/reports/attendance.csv`	Scoped staff
Policy	GET/PUT `/policy`	Authenticated read; administrator write with reason

Signing protocol

`POST /attendance/verify` and `/biometrics/enroll` accept the three required fields below and optional App Attest assertion/key fields:

```
{
  "payload": "<exact serialized Evidence JSON string>",
  "signature": "<base64 DER ECDSA/SHA-256 signature>",
  "idempotency_key": "<16 to 64 characters: letters, digits, hyphens or underscores>"
}
```

P-256 ECDSA/SHA-256 signs the bytes of `idempotency_key + "." + payload`; the server verifies with the registered public key. Legacy devices retain the original hexadecimal HMAC protocol during migration. The inner `Evidence` schema has challenge/device IDs, 8–40 timestamped JPEG frames, consent, optional location, real Wi-Fi observations/status and optional integrity token. Coordinates, accuracy and timestamp are measurements; no `distance`, `inside_geofence`, `similarity` or `liveness_passed` field is accepted as proof. Clients should keep the same exact signed request and key after a network timeout to recover the original attendance decision.

Enrollment and attendance purposes are not interchangeable. An enrollment challenge cannot reference a session; an attendance challenge must bind an active, enrolled session and an existing biometric profile. An exact authenticated enrollment retry returns the recorded success while the profile exists; a different request cannot reuse a consumed enrollment challenge. The client can also query `/biometrics` for current status.

Outcomes and errors

An attributable attendance attempt contains `id`, `state`, `reasons`, `risk_score` and `evidence_summary`. `PRESENT` creates one official record. `PENDING_REVIEW`, `REQUIRES_RETRY`, `REJECTED` and `EXPIRED` do not create a present record. Faculty may create an explicitly sourced correction later.

HTTP errors use `{"error":{"code":403,"message":"..."}}`. Validation responses include field locations without echoing sensitive input. Typical codes are 401 authentication failure, 403 role/device/signature denial, 404 missing resource, 409 conflict/replay, 413 body limit, 422 invalid evidence, 429 rate limit and 503 missing verification service/models. Never treat an HTTP failure as an attendance approval.

Enrollment vision failures additionally return optional `error.details` with allowlisted stage/count fields: `completed_actions`, `expected_action`, `frame_count`, `missing_landmark_frames`, and `frame_index`. No image, landmark coordinate or embedding is returned. The client uses these fields for targeted retry instructions. Rate-limit `Retry-After` reports the remaining bucket window. Definitive 4xx rejections discard the saved request; network/5xx failures can retain the same signed submission for idempotent recovery.

The mobile sign-in client now translates `body.email` and `body.password` validation fields into specific guidance and rejects embedded email whitespace before submission. The server's validation envelope is unchanged; no password values are echoed. HTTP 401 authentication failures remain distinct from format errors.

Review lifecycle: creating a student dispute does not update attendance. `POST /corrections` creates/updates an official record with source CORRECTION and an attributable reason/audit entry. `POST /disputes/{id}/resolve` closes the dispute but does not create an attendance record by itself. The original attempt decision is retained after both operations.

Most operational lists expose the latest 500 rows; course and enrollment lists are not paginated. CSV export includes all records in the authorized selection. Large institutions need cursor pagination, query optimization and retention jobs before deployment.

Cross-platform completion update

`POST /devices/registration-challenge` issues a one-use nonce. `POST /devices` accepts public_key (base64 canonical P-256 SPKI), registration_challenge_id, registration_signature and reported key_storage in addition to password/label/platform. New registrations do not return a shared secret. See CROSS_PLATFORM_IMPLEMENTATION.md for exact signing bytes. `GET /attestation/capabilities`, `POST /attestation/challenge`, and `POST /attestation/register` implement the App Attest boundary. Signed requests may contain outer attestation_key_id and attestation_assertion; the assertion covers the exact idempotency/payload bytes. Evidence adds accuracy_authorization, wifi_source and current_bssid. `PUT /policy` accepts ios_presence=review|retry|dynamic with an audit reason. `GET /admin/devices` is administrator-only and explicitly labels reported storage versus verified app registration. The regenerated openapi.json is the field-level contract.

Biometric design

Current attendance also requires the separate QR/audio presence proof. It does not replace face matching or liveness; enrollment remains its own consented flow. Attendance challenge lifetime is now 120 seconds to include the coordinated presence exchange; enrollment retains the configured lifetime (normally 60 seconds).

Model provenance and separation of responsibilities

YuNet detects a face and five alignment landmarks. It does not establish identity. SFace generates a 128-dimensional learned face representation, using OpenCV's `FaceRecognizerSF.alignCrop` and `feature` APIs. MediaPipe Face Mesh separately supplies dense facial landmarks for temporal liveness. ONNX models are executed by OpenCV DNN on CPU; ONNX Runtime is unnecessary for this pipeline.

Asset	Provenance	License
`face_detection_yunet_2023mar.onnx`	OpenCV Zoo, commit `48b10fa505aeb51fbd127fc60a5c660040e823b7`	MIT; Shiqi Yu notice retained
`face_recognition_sface_2021dec.onnx`	Same official repository/commit	Apache 2.0; full license retained
`face_landmarker.task` float16 v1	Official Google MediaPipe model storage; consumed by MediaPipe 1.1.1 Tasks	MediaPipe code is Apache 2.0; consult the linked official model card/provider terms for the pretrained asset

Expected SHA-256 values are in `scripts/download_models.py`, copied from the official Git LFS pointer objects. A mismatch stops installation. The SFace asset is 38,696,353 bytes and YuNet is 232,589 bytes. No model is trained in this project.

The landmark task is downloaded over verified HTTPS from the versioned official Google URL. Its SHA-256 is pinned as `64184e229b263107bc2b804c6625db1341ff2bb731874b0bcc2fe6544e0bc9ff`; it is loaded in IMAGE mode with at most two faces. The current MediaPipe Tasks API avoids the legacy package's protobuf dependency.

Enrollment and comparison

Enrollment requires an authenticated student, an authorized device, explicit consent, a fresh enrollment-purpose challenge and a server-validated sequence. JSON cannot carry a trusted embedding or similarity score. JPEG input is base64-decoded, limited to dimensions 160–1280, checked for brightness and Laplacian sharpness, and must contain exactly one usable YuNet face of at least 70 pixels. The aligned BGR crop passes through OpenCV's SFace preprocessing. Returned vectors must contain exactly 128 finite values and have nonzero norm. They are L2-normalized.

Every verification frame is embedded and compared against the encrypted enrolled template. Enrollment frames must match the first frame so that a sequence cannot trivially switch people. The minimum similarity across frames must meet `FACE_THRESHOLD`, default **0.5**. OpenCV's tutorial gives a cosine baseline of approximately 0.363 for its example evaluation; 0.5 here is a stricter initial operating point, not a locally calibrated false-acceptance guarantee. A realistic institution must collect consented evaluation data and measure FAR/FRR across lighting, pose and user groups before choosing a threshold.

The enrollment template is the normalized mean of sequence embeddings and is encrypted with Fernet under a separate `BIOMETRIC_KEY`. Templates carry an explicit model version. Raw frame bytes and unencrypted embeddings are not persisted. Student deletion requires password confirmation. Backup retention can still retain deleted ciphertext; operators must apply their retention policy to backups as well.

Liveness protocol

The server issues an unpredictable UUID challenge bound to a student, device, purpose and attendance session (when applicable). It selects a left/right turn and randomizes the turn/blink ordering, inserting center poses. Attendance challenges expire after 120 seconds; enrollment uses the configured lifetime, normally 60 seconds. Challenges are consumed once, and attempt/issuance budgets remain rate limited.

Face Mesh computes eye aspect ratio (EAR) from six landmarks per eye, averaged across both eyes. A blink must transition open (`EAR > 0.23`) → closed (`EAR < 0.19`) → open. Head direction is a signed nose displacement relative to eye centers, normalized by interocular distance. Center is absolute displacement below 0.12; turns exceed 0.18 in the requested direction. Two consecutive observations must support each non-blink pose. This is a heuristic displacement proxy rather than a calibrated three-dimensional head-pose estimator.

Turn names use the student's own left/right. For unmirrored captures, nose movement toward image-right maps to `TURN_LEFT`; the displacement sign is inverted before evaluating the action. The client gives 700 ms to respond to a new instruction and keeps each blink phase visible across completed photos.

At most two frames may lack MediaPipe landmarks, with at least eight tracked frames still required. Missing landmarks do not supply liveness evidence and reset any incomplete pose/blink. Every frame, including an untracked frame, must still pass image quality, single-face detection and identity comparison. Persistent tracking loss rejects the sequence. Enrollment errors return only stage/count diagnostics to provide specific retry guidance; no images, landmark coordinates or embeddings are returned or logged.

A sequence contains 8–40 frames, has increasing timestamps separated by 0.08–2.5 seconds, spans at least two seconds, remains inside the challenge time window, and may not claim a final capture more than two seconds in the future. Identical encoded frames are rejected. Mobile captures unmirrored camera images and synchronizes capture timestamps to the challenge's server clock. Direction conventions, timing and EAR thresholds require physical-camera validation; basic frame rates can miss a fast blink, so the UI asks for a slow close/open action.

Limits and measured evidence

Landmark liveness rejects a static frame and some naive replay attempts. It does not prove camera provenance and can be defeated by sophisticated replay, virtual-camera injection, deepfakes or a compromised client. Randomized action order and signatures increase the effort but are not certified presentation-attack detection. Client timestamps are assertions checked for consistency, not trusted hardware timestamps.

Real model tests use the public NASA astronaut image from scikit-image and Grace Hopper image from matplotlib: same-face photometric variation, different-person comparison, zero faces and multiple faces. These smoke tests validate model operation; four tests on two portraits are not a biometric accuracy study. No live student was enrolled during automated evaluation.

Cross-platform completion update

Both platforms submit the same processed unmirrored JPEG format and randomized action order. The client cancels backgrounded capture, checks challenge expiry before/after each frame, deletes temporary images and translates quality/no-face/sequence errors into retry guidance. The server rejects non-finite landmark geometry and tests dark/overexposed frames using the real inference pipeline. Do not infer physical-camera frame rate or left/right orientation from JS export success. Calibration must use separate consented development and held-out evaluation participants; stratify lighting, eyewear, skin tones, device/OS and accessibility needs. Measure false match/non-match rates with confidence intervals and failure-to-acquire, not a single invented accuracy figure. Keep raw consented evaluation data outside Git with access controls, deletion dates and a non-biometric review route; freeze thresholds only after review.

Security architecture

Dynamic presence proof boundary

Policy v2 requires signed QR/audio evidence bound to the student, device, attempt, session, classroom and current display epoch. QR expiry, single-use audio answers, server deadlines, serialized playback and idempotency are server enforced. The faculty playback endpoint, not the student API, exposes the current audio payload. Raw microphone PCM remains transient on-device.

These controls do not attest capture provenance or physical distance. Current QR and sound can be recorded/shared/relayed; a five-second deadline is not distance bounding. Stop/restart invalidates proofs, and mandatory identity/location/required attestation rules remain. See the protocol and physical leakage/relay tests. Historical successes must not be retroactively attributed to v2.

Authentication and authorization

Passwords use Argon2id with memory cost 65,536 KiB, time cost 3 and parallelism 2. Failed logins use a constant-work password check against a dummy hash when the account is absent. Tokens are HS256 JWTs with explicit issuer, audience, type, issued-at, expiry, subject and random ID; the secret must be at least 32 characters and is randomly generated by setup. Access lifetime defaults to 15 minutes. Refresh tokens are random 48-byte URL-safe values stored only as SHA-256 hashes, with a default seven-day lifetime. Rotation is transactional; reused tokens revoke their refresh family.

Logout revokes refresh sessions. Already-issued access tokens remain valid until expiry unless the account is disabled; endpoints check active status and current role on every request. This access-token revocation window is documented, not concealed. Role updates revoke refresh tokens and device bindings and are administrator-only with reasons. The dashboard keeps credentials in memory, so reload requires login. Mobile credentials use OS secure storage.

Device binding

New registration verifies a password and one-use nonce proof from a native P-256 key. The private key remains inside Android Keystore or iOS Secure Enclave (Keychain for simulator); only canonical SPKI public-key bytes reach the server. Evidence signs exact UTF-8 `idempotency_key + "." + payload` with ECDSA/SHA-256 and a base64 DER signature. Existing HMAC devices remain compatible; new legacy registrations are disabled by default.

The database has a partial unique active-device index. Approved reset revokes old devices; reinstall/key loss cannot silently create a replacement authorized key. Reported hardware backing is not independent key attestation. App Attest verifies a configured pinned chain, app ID, nonce, environment and public key, then checks assertions with a strictly increasing counter. Genuine Apple interoperability and physical-device evaluation remain outstanding. Play Integrity remains unconfigured and fails closed. Never forward a client boolean as a verdict. Full setup and verification limits are in CROSS_PLATFORM_IMPLEMENTATION.md.

Evidence and decisions

Identity gates precede presence scoring. Enrollment, session, face, challenge and device failures cannot be overridden by a good GPS score. Server-owned rules calculate distance and signal match. A fully outside uncertainty circle rejects; an overlapping boundary requests review. Missing/inaccurate/stale data requests retry or follows the explicit Wi-Fi policy. Mock-location flags reject when reported, but the absence of a flag does not prove an honest OS.

The verified request atomically consumes its challenge, stores an attempt and optionally inserts one attendance record. Retries with the same user/key and exact payload return the original attempt; modified payloads conflict. A different key cannot reuse a consumed challenge. Database locks serialize concurrent retries. Manual corrections are distinct records

with actor, reason and before/after status.

Storage and operations

Fernet protects templates and signing credentials at rest with a dedicated key. Secrets, caches and generated local state are excluded from archives. There is no raw-image retention endpoint. Location summaries are coarse and restricted by role. Production operators should use separate least-privilege migration/runtime database accounts, secret management, key rotation with re-encryption, a retention process, encrypted backups and trusted TLS termination.

Rate buckets live in PostgreSQL, shared across workers. The API does not trust caller-supplied forwarded headers for rate identity; behind a reverse proxy, connection IP limits can group multiple clients. Account limits remain independent. Configure trusted proxy handling and infrastructure limits for deployment. Bounded request bodies, decoded image dimensions and model serialization reduce resource abuse but do not replace load testing or upstream request throttling.

Audit rows are append-only through ordinary application APIs except their reviewed flag. A database administrator can still alter them: this is not tamper-proof external audit storage. Implement a protected external append-only sink if that is an institutional requirement.

Cross-platform completion update

New clients use native non-exported P-256 keys and proof of possession. Android hardware backing is queried locally; physical iPhones require Secure Enclave and simulators use device-only Keychain. Storage metadata alone is not server-verified hardware attestation. App Attest verification enforces a configured pinned Apple root, app identity, nonce, environment, key binding and increasing assertion counter. Synthetic certificates verify logic only; actual Apple-provider acceptance remains unverified. Android Play Integrity remains fail closed. Legacy HMAC registration is disabled by default, existing keys migrate through audited resets. Exact enrollment retries recover a lost HTTP response without permitting different replay payloads. Required attestation cannot be replaced by a favorable GPS score.

Policy v2 retires the iOS gps_only option. Current iOS choices are review, retry and dynamic; the migration maps an existing gps_only choice to dynamic while retaining stricter review/retry settings. Dynamic requires completed QR/audio proof alongside mandatory identity/device/GPS gates; a global requirement for attestation still applies. The local review policy was not changed. QR/audio does not independently attest a platform or sensor and can be relayed. See the protocol.

Threat model

QR/audio extension

The current flow adds expiring QR tokens and a one-use per-attempt acoustic answer available to scoped faculty playback controls. Epoch invalidation, exact signatures, server-clock deadlines and consumption address replay/state confusion. Live relay, current-window recordings, sound leakage, compromised clients and faculty-key misuse remain possible. This is not a distance-bound or independently attested proximity proof. Physical speaker/microphone, noisy-room, doorway/adjacent-room and accessibility trials have not started; see the matrix.

Assets include student identity, biometric templates, device credentials, location evidence, attendance integrity, refresh sessions and faculty audit accountability. Trust boundaries are the mobile device to HTTPS API, staff browser to API, API to PostgreSQL, and operator secrets to server runtime. A student client is untrusted even after login.

Threat	Mitigation in this implementation	Residual risk / evaluation
Shared QR/token	No standalone shared token can create attendance; enrolled identity, challenge and device are required	Account and device credential collusion remains possible
Face photo	Actual embedding plus randomized temporal eye/head sequence	Advanced replay/deepfake/injection can mimic landmarks
Client says liveness passed	Schema rejects unknown verdict fields; server analyzes frames	Frame capture provenance is not attested
Hostel or remote attempt	Server GPS geofence, accuracy, age and available Wi-Fi	Compromised phones may forge all signals
GPS spoofing	Mock indicator, stale/accuracy checks, coarse travel anomaly	An omitted/forged mock flag is not trustworthy
Device ID cloning	Native P-256 proof and signed evidence; one active binding; legacy HMAC compatibility	Legacy secrets can be copied; native storage reports are not independent key attestation
Replay or duplicate	Purpose/user/device/session binding, expiry, single use, unique keys and records	Encoded video may be re-rendered to evade byte equality; liveness is basic
Concurrent submission	Student/challenge/session locks and unique constraints	Session-level serialization limits throughput
Faculty accesses another class	Server course-ownership checks and query scoping	Privileged administrator has institution-wide access
Silent attendance edit	Mandatory reasons, correction ledger, structured events	Database-superuser tampering is outside app controls
Refresh-token theft/reuse	Secure storage, hashing, rotation, family revocation	Access tokens have an expiry window; browser XSS can act while logged in
CSV formula injection	Escape dangerous leading formula characters	Downstream spreadsheet handling still needs care
Large payload/decoded image	Actual-byte body bound, JPEG byte and dimensions cap	Valid expensive requests still consume inference capacity

Threat	Mitigation in this implementation	Residual risk / evaluation
Dependency compromise	Lockfiles, verified model hashes, audit reports	Audits are time-bound and cannot prove absence of defects

Abuse evaluation protocol

Use consented volunteers on both real Android and iPhone devices. Test identical-photo presentation, screen replay, a wrong person, multiple faces, same evidence/key replay, same evidence/new key replay, a second registered device, a revoked device, a distant location, disabled location permission and unavailable Wi-Fi. Inspect both HTTP outcomes and resulting database/audit records. Record model version, threshold, device model, OS version, illumination and capture timing. Never label a human review or an unrun test as successful automatic presence verification.

The public test portraits are used only for model smoke tests. No external biometric service receives student images, and this implementation does not use facial recognition for surveillance or sensitive-trait inference.

Cross-platform completion update

New threat cases: App Attest development/production confusion, attacker-selected trust roots, certificate nonce substitution, assertion-counter replay, app reinstall with persistent iOS Keychain entries, and presenting iOS BSSID identity as multi-AP RSSI evidence. Controls include server-configured root fingerprint/app ID/environment, challenge consumption and key uniqueness, atomic counters, installation-ID checks, and separate server iOS policy. Synthetic adversarial tests cover protocol rejection; physical sensor injection, rooted-device behavior and sophisticated presentation attacks still require evaluation. Keychain-only simulator keys are never claimed to be Secure Enclave keys.

Policy v2 retires the iOS gps_only option. Current iOS choices are review, retry and dynamic; the migration maps an existing gps_only choice to dynamic while retaining stricter review/retry settings. Dynamic requires completed QR/audio proof alongside mandatory identity/device/GPS gates; a global requirement for attestation still applies. The local review policy was not changed. QR/audio does not independently attest a platform or sensor and can be relayed. See the protocol.

Current and historical test/build results

Current local checkpoint - 27 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Backend	146	0	0	JUnit
Mobile	64	0	0	JUnit
Dashboard	11	0	0	JUnit
Native Wi-Fi	10	0	0	JUnit

231 local tests passed. TypeScript, dashboard build, both JS exports, migration/model drift, generated native configuration, Android arm64 APK/signature, iOS simulator and signed-device builds passed. SpvsAcoustic is present in the new APK and the iOS native compilation. No rebuilt phone installation or physical QR/audio test was performed. See local results.

The upstream cloud ran three browser tests and reports 234 total. Those browser and container checks were not rerun against the local demo database, and are separate from the local total. Fresh npm audits report four high mobile affected entries and zero dashboard findings; backend vulnerability scanning is historical.

Earlier C003 automatic inside attendance and the later hostel GPS rejection precede policy v2. See C003 validation. All further physical testing waits for explicit user readiness in the classroom.

Prior local checkpoint - 26 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Full backend	120	0	0	JUnit
Mobile	46	0	0	JUnit
Dashboard	8	0	0	JUnit
Native Wi-Fi	10	0	0	JUnit

184 passed, all rerun for this checkpoint. Mobile TypeScript and dashboard production build passed. The APK/native build remains the verified 24 October development host; no native code changed, and current JavaScript is served by Metro. The physical Android demo reached manual faculty API correction, resolved dispute, attributable audit and verified phone history. Automatic classroom approval and faculty-browser click-through remain unverified. See physical evidence.

Fresh npm audits: mobile has 5 high affected package entries, dashboard has zero. Backend security auditing remains historical. Prior browser/container checks and native build runs retain their original dates; repeated tests are not added to this total. See current verification.

Prior local checkpoint - 24 October 2026

Suite	Passed	Failures/errors	Skipped	Evidence
Backend, full suite	120	0	0	JUnit
Mobile	42	0	0	JUnit
Dashboard	8	0	0	JUnit
Android native Wi-Fi	10	0	0	JUnit

Total: **180 passed** across these four suites. The backend/mobile/dashboard results were rerun for the audit; the native results are from the preceding verified local Android build. Historical browser tests are excluded from this total. Mobile TypeScript and the dashboard production build passed. Native Android compilation and APK signature verification passed; physical Android testing is pending. See Android validation.

iOS simulator and signed-device builds passed in the earlier local Mac run. Physical iPhone enrollment succeeded on 24 October; an attendance submission passed face/liveness/GPS but required review for iOS presence policy. No official attendance record was created. See iOS validation.

Fresh npm audits: mobile reports 5 high affected package entries; dashboard reports zero. Backend vulnerability auditing was not rerun locally. Upstream deprecation warnings and genuine provider-attestation/calibration gaps remain.

See the full audit for scope and provenance.

Historical cloud completion run - 23 October 2026

The tables and statements in this section describe that earlier run only.

Suite	Passed	Failures/errors	Skipped
Backend	109	0	0
Mobile	28	0	0
Dashboard	8	0	0
Browser	3	0	0

Total: **148 passed**. See FINAL_COMPLETION_REPORT.md for test boundaries, commands and intermediate failures. Baseline evidence is retained separately.

Capability	Observed status
Android debug APK	PASS: genuine Gradle assembleDebug, arm64-v8a; debug signed, development client; physical-device testing NOT RUN
Android/iOS JS export	PASS: separate platform exports; final combined export in evidence
iOS native project	PASS: Expo prebuild, camera/location/privacy settings, local podspec/autolinking and static configuration checks

Capability	Observed status
iOS native compilation / simulator / IPA	NOT RUN: Linux host; no macOS/Xcode or Apple signing identity
PostgreSQL migrations	PASS: fresh upgrade, check, full downgrade, re-upgrade, check on isolated PostgreSQL 16 database
Backend and dashboard containers	PASS: both images built and used for real browser/API tests
Physical Android / physical iPhone	NOT TESTED
Apple App Attest	Synthetic verifier/counter tests passed; genuine provider/device verification UNVERIFIED
Google Play Integrity	NOT CONFIGURED; fail-closed boundary only
Biometric calibration	NOT PERFORMED; six real-model smoke cases are not accuracy metrics

No physical Android/iPhone, iOS simulator, Xcode or genuine provider-attestation result is claimed. Synthetic API/sensor/certificate inputs and mocked React Native components are identified in their test modules.

Requirements traceability

Requirement	Implementation	Automated evidence	Remaining validation
Persistent normalized storage	20 SQLAlchemy tables + explicit Alembic migration	Migration/table/constraint tests on PostgreSQL	Deployment backups and restore drill
Password/JWT/refresh security	`core/security.py`, `api/auth.py`	Login, invalid JWT, rotation, reuse and logout tests	Operational key rotation
Backend role restrictions	`roles`, `access_course`, `session_ids`	Student/staff denial, cross-course checks and role-change tests	Institution-specific account lifecycle
Academic management	`api/academic.py`, dashboard courses/classrooms/sessions	CRUD/session lifecycle and browser workflow	Real campus data entry
Actual face recognition	YuNet + SFace in `services/vision.py`	Real same-face/different-person/no-face/multi-face tests	Consented local calibration
Enrollment consent/deletion	`/biometrics/enroll`, DELETE `/biometrics`	API consent/storage/deletion contract tests	Live student enrollment
Temporal liveness	MediaPipe landmarks + sequence validator	EAR/pose sequence, timing, expiry tests	Physical phone spoofing/usability matrix
Device binding	Native P-256 keys + nonce proof + unique active-device index	Signature/device mismatch, second-device denial and reset tests	Physical Keystore/Secure Enclave evaluation
Platform integrity	App Attest native/server verifier; Play Integrity fail-closed boundary	Unconfigured policy never passes as verified	Genuine provider accounts, physical-device interoperability and receipt risk evaluation
GPS evidence	Haversine + uncertainty/freshness gates	Boundary/stale/inaccurate/mock/travel tests	Indoor GPS field measurements
Wi-Fi evidence	Native Android scan module + reference matching	Matching/weak/unavailable policy tests	Kotlin build and real AP scans; see current report
Deterministic decision	`/attendance/verify` + presence service	Hard-gate and decision API tests	Full real-device happy path
Concurrency/idempotency	Transaction locks + unique constraints	Simultaneous identical requests, replay and duplicates	Load testing
Auditing/anomalies	Structured events and correction ledger	Repeated failures, device reset, policy/role/correction audit	External immutable sink if required
Faculty reports	Aggregates, Recharts and scoped CSV	CSV escaping, live browser export, actual API counts	Large data export performance
Student disputes	Mobile history + scoped dispute API	Ownership and resolution tests	Real student support process
Dashboard quality	React, router, query, labeled forms and responsive CSS	Interaction tests, production build, Chromium workflow	Assistive technology and usability study

Requirement	Implementation	Automated evidence	Remaining validation
Student app integration	Expo camera/location/SecureStore/API signing	TypeScript, HMAC/boundary tests, Metro/Hermes export	Physical sensor/permission tests
Documentation/artifacts	Markdown, diagrams, PDF/ZIP packaging script	PDF text/page and ZIP CRC/contents checks	Fill academic identity placeholders

“Automated” in this table does not imply manually tested. The generated verification report identifies the current counts and commands. No fake biometric approval or sensor measurement is used by application code.

Cross-platform completion update

Added requirement	Implementation	Verification boundary
Android/iOS native device keys	spvs-security, device_keys.py, registration nonce	Real crypto/API tests; physical key storage untested
iOS presence policy	ios_presence, current_bssid, accuracy_authorization	PostgreSQL/API/unit and admin browser tests
App Attest	Swift DeviceCheck bridge, app_attest.py, key/counter tables	Synthetic chain/assertions; genuine provider setup outstanding
Cross-platform camera state machine	capture.ts, shared UI and error guidance	Logic/component tests; real capture untested
Reproducible native projects	config plugin, podspecs, macOS build script	Prebuild/config/autolinking; native outcomes in final report
Enrollment recovery	enrollment_request_hash	Exact retry and different-key replay tests

Known limitations and release boundary

The current local status supersedes historical build and test claims. Xcode/simulator and signed-device builds have run. One physical iPhone enrollment and one pending-review attendance attempt are recorded in iOS validation. The Android hostel demo now covers scans, registration, enrollment, submission, manual correction and phone history. Actual classroom discrimination, broader device tests and genuine Apple App Attest remain outstanding.

The 27 October update builds the new mandatory QR/audio clients locally and migrates the database, but no physical QR/audio test has started. Earlier automatic C003 attendance predates that flow.

- QR/screenshots and acoustic responses can be shared or relayed within their

live windows; this is not distance-bounding or cryptographic proximity.

- Audio playback is serialized by five-second reservations (at most 12 individual

challenges/minute). Large-class throughput and classroom audio behavior are unmeasured.

- Speaker/microphone response, noise, interruptions, accessibility, leakage through

doors/walls and real-time relays need the documented physical test plan.

- Old installed native apps lack SpvsAcoustic. Rebuilt clients must be installed

before testing; no installation or new trial occurred during this sync.

Verification assurance

- A normal phone can be rooted, instrumented or used with injected camera/location data. A native device signature (or a legacy HMAC) proves key possession, not trustworthy physical capture.

- Temporal landmark liveness is a heuristic. It is not ISO-certified presentation-attack detection, and sophisticated replay or deepfake attacks may succeed. The action vocabulary is small even though challenge IDs and ordering are randomized.

- SFace is a real pretrained model, but the chosen threshold has not been calibrated on this institution. The public-portrait smoke tests do not estimate FAR, FRR, fairness or operational accuracy.

- The corrected capture flow worked for the observed iPhone and Motorola enrollments; the Motorola had one IMAGE_QUALITY_LOW rejection before success. Sustained usability, both turn directions across phones, and broader device/user evaluation remain unverified. Automated API tests substitute vision and do not replace hardware evaluation.

- GPS and Wi-Fi support approximate physical presence, not absolute classroom proof. Rounded location history supports only coarse anomaly rules. The absence of a mock-location flag does not establish honesty.

- Wi-Fi scans are implemented only through a local Android module and depend on OS permissions, enabled location, recent scan availability and throttling. iOS may provide a permitted current BSSID only; no RSSI scan is fabricated. The separate default iPhone policy requires review.

- Play Integrity remains an unconfigured provider boundary. App Attest has native and server code, but Apple configuration and genuine provider/device validation remain outstanding. Requiring unverified attestation moves otherwise eligible attempts to review.

Application and operational limits

- The hostel minimum-RSSI experiment accepted both inside and corridor samples

under identical synthetic valid GPS, with stronger signals outside near the AP. This is not actual classroom calibration; no hostel thresholds were installed.

- Later C003 sampling produced a provisional fit for two 5 GHz references at

-70 dBm and one older-flow automatic inside pass. The attempted outside attendance was actually from the hostel and stopped at GPS. Fresh near-door Wi-Fi-specific validation was not completed; see C003 evidence.

- WIFI_UNAVAILABLE can mean missing AP references even when the phone supplied a

scan. More specific configuration-versus-sensor reason codes remain a UX item.

- After manual correction, original attempts remain PENDING_REVIEW for audit.

The history UI can retain generic pending wording/review actions; official records are authoritative. The incoming mobile API now adds bounded request waits, but its physical network-interruption behavior remains to be tested.

- The latest faculty review test exercised the authenticated demo API and phone

history, not a complete faculty-browser click-through or institutional approval.

- Web lists generally show the latest 500 entries. Large datasets need cursor pagination and indexed query tuning; exports can be large.

- Session-row locks serialize verification and manual correction within a session. This prioritizes consistency and requires throughput evaluation before large lectures.

- Inference is CPU-based and serialized per worker. Model memory is repeated across workers; no measured capacity or latency SLA is claimed.

- Dashboard access tokens live in memory. Reload requires login. Logout revokes refresh tokens; existing access tokens have a maximum 15-minute default expiry window unless the user is disabled.

- Exact signed enrollment retries recover a successful response; different replay requests conflict. Attendance retains durable idempotency.

- Audit records are protected through application authorization, not an external tamper-proof log. A database administrator can modify storage.

- No automated retention purge, key rotation/re-encryption job, institution SSO, password reset/email service or external audit export sink is included. Identity/role provisioning is institution-managed in the dashboard.

- In a real college, biometric consent, an accessible alternative attendance process, retention and dispute policies require institutional decisions. This software does not supply those policies.

- Demo Compose uses a database-owner account, local development secrets and no TLS reverse proxy. Harden deployment according to `DEPLOYMENT_GUIDE.md` before using real student data.

Dependency audit boundary

The 27 October npm audit reports **4 high-severity mobile package entries** and zero dashboard findings. Direct advisory records identify `node-forge` (GHSA-86w9-cpqp-85rv) and `zod` (GHSA-mrhx-c7xc-m3w5); the other affected entries are dependency propagation. See the saved mobile audit. The earlier 15-entry/braces result remains historical. Locked dependencies were refreshed for the incoming feature; no automatic advisory fix was applied. npm still proposes an incompatible Expo 44 downgrade for parts of the tree. Backend vulnerability scanning was not rerun; its older zero-finding report is not a current clearance. Keep development tooling private and review compatible fixes before distribution.

See the verification report for exact build, native compilation and physical-device test status. An exported JavaScript bundle is not an installed/tested Android APK, and Linux validation is not macOS ARM64 validation.

Historical native build blocker

The baseline release failed downloading the React Native Android AAR with HTTP 403. This completion explicitly configures the managed proxy and CA trust, retaining TLS verification. The latest actual build outcome supersedes that baseline and is recorded in FINAL_COMPLETION_REPORT.md. JavaScript exports are reported separately from native compilation and physical-device testing.

Historical cross-platform completion update (23 October cloud run)

The current native build outcomes are in FINAL_COMPLETION_REPORT.md. No physical Android or iPhone run, Xcode compilation, simulator interaction, Apple Silicon execution, genuine Apple attestation registration, Play Integrity provider verification, institutional deployment or population biometric calibration is claimed. App Attest certificate/assertion logic is implemented and synthetic-tested; Apple receipt risk analysis is not implemented. CocoaPods resolution awaits macOS; no validated Podfile.lock is invented. `braces` <=3.0.3 and `node-forge` <=1.4.0 remain affected upstream; official registry latest versions were still 3.0.3 and 1.4.0 during this run. npm reports 15 affected mobile package entries from these two root advisories. Do not use npm force-fix downgrades to incompatible Expo 44/React Native 0.72. Keep Metro/dev tooling private, build only trusted source and do not use its vulnerable certificate utilities as an institutional trust authority. Backend and dashboard audit results are recorded separately.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

SPVS current completion and verification report

Checkpoint: 27 October 2026, Asia/Kolkata. Integration source: `0fc235f916fb662db71c298091255ec10f0917f6`. Publication: PR #7. It incorporates upstream main `404eed84cbb9f5f2eb0bc73e7b66910fb49476f9` (QR/acoustic PR #6). Artifact metadata and FINAL_FILE_MANIFEST.txt identify the exact delivery snapshot.

Incoming feature and local integration

The new attendance flow requires rotating classroom QR and individual acoustic proof in addition to the existing device, face/liveness, GPS and platform-presence requirements. Faculty starts/stops challenges, displays QR and plays the queued student's signal. Student microphone capture is bounded/foreground; PCM remains transient on the phone. QR rotation is five seconds, validity at most ten seconds, and audio redemption has a server-clock five-second deadline.

The local main was clean and two commits behind, so it was fast-forwarded. Dashboard dependencies were refreshed from the lockfile. A private local database backup was created and checked; migration `9bd32a88f101` and model-drift check passed. Existing accounts, devices, profiles, sessions, attempts, records, corrections, disputes and C003 references were preserved. Local iOS and unavailable Wi-Fi policies remain review; no physical session challenge was enabled.

API/dashboard were restarted; health, model readiness and the new presence route were verified. Android/iOS projects were regenerated, microphone declarations checked, and the resolved Podfile.lock now includes SpvsAcoustic.

Current local evidence

Check	Result
Backend	146 passed
Mobile	64 passed; TypeScript passed
Dashboard	11 passed; production build passed
Native Wi-Fi	10 passed
Total local	231 passed, no failures/skips
Android/iOS JS exports	Passed
Android native APK	Built arm64; acoustic class present; signature passed
iOS simulator	Xcode build passed; acoustic Swift source compiled
iOS signed device app	Build and codesign verification passed
Migration/configuration	Upgrade, model drift and native declarations passed

Evidence is in local-2026-10-27. The upstream cloud reports three additional browser tests (234 total in that environment); they were not rerun against the local live demo database and are not included in the local total. Cloud container checks remain separate historical evidence. Unit tests include synthetic PCM and mocked native/UI inputs, not real sound interoperability. Upstream deprecation warnings remain.

The new APK is 78,138,028 bytes, SHA-256 `6cd0d397cccbbc42f6033c9a98eeab5a7c0d2f3f3d312b9dfdef72c607ab35c2`. It replaces the old APK lacking SpvsAcoustic. No rebuilt app was installed or launched on either phone during this update. The signed iPhone app stays in ignored local build output; no production IPA or private signing material is published.

Physical evidence before policy v2

Earlier iPhone registration/enrollment and the Motorola hostel demo through manual faculty API correction, dispute resolution and verified history remain valid observations of the preceding flow. They are not physical QR/audio results.

C003 was configured separately with the user's coordinates, a 50 m radius and two provisional 5 GHz references at -70 dBm. Three inside diagnostic samples and one closed-door corridor sample informed this provisional fit. The existing half-match rule accepts one of two references; no independent router mapping or statistical calibration was claimed.

Inside C003 on 26 October at 18:34:48 IST, the old flow returned automatic PRESENT, source VERIFICATION, with Wi-Fi ratio 0.5 and eligible face/liveness/GPS. The later attempt at 23:57:53 IST was confirmed by the user to be from the hostel. It returned OUTSIDE_GEOFENCE at reported distance 151.1 m and created no record. The geofence returned before Wi-Fi matching, so this does not validate near-door Wi-Fi exclusion.

Full measurements, location correction and evidence scope: C003 validation.

Remaining gates and testing hold

All further physical tests wait for explicit user readiness in the classroom. The rebuilt native clients must be installed, and the faculty must enable a fresh session's QR/audio controls. Use the physical plan. Record the gate determining a failure; failed QR/audio/GPS does not isolate Wi-Fi.

Pending: real speaker/microphone interoperability, permissions/interruptions, timing under network load, doorway/adjacent-room sound leakage and real-time relay, accessible alternatives, new-flow automatic classroom decisions, broader biometric and device calibration, genuine provider attestation and production operations.

QR/audio can be shared or relayed. The serialized five-second playback reservation limits the channel to at most 12 individual challenges per minute. This is not distance-bounding, cryptographic proximity or production capacity validation.

The local iOS review policy was preserved. The migration retires legacy gps_only to dynamic where present; dynamic is a separate audited opt-in and does not override mandatory identity/location or required attestation gates. QR/audio is not App Attest.

Fresh npm audit: 4 high mobile affected entries; dashboard zero. Backend security auditing remains historical. No forced downgrade or general security clearance is claimed. Historical history/review wording issues remain documented; the new mobile API adds bounded request waits, which are not yet physical-network validated.

Delivery

Current source ZIP, project/setup PDFs, packaged Markdown, milestone PDFs and checksums are regenerated together. The archive includes QR/audio code, migration, native locks, sanitized C003 evidence and local test/build records. CRC/member, source hash, artifact hash, PDF readability and link checks are performed.

Secrets, private backups/databases, raw AP identifiers, precise locations, keys, biometric templates, raw frames/audio and raw screen captures remain excluded. Old dated reports and Git history preserve prior results. Source publication does not deploy a cloud service or establish institutional production readiness.

C003 classroom calibration and attendance observations

Record date: **27 October 2026, Asia/Kolkata**. Physical observations below were made on 26 October, before the local QR/acoustic policy-v2 runtime update. They do not establish physical operation of the new QR/audio flow.

Classroom configuration

Created a separate **120 seater C003** classroom using the user's Maps coordinates, retaining a 50 m GPS radius. The earlier hostel test room remains separate so its history keeps the original configuration. Precise coordinates, database IDs, SSID and raw BSSIDs remain local and are excluded from this public record.

The user reports two physical routers and a door at the far-left front. Mapping the observed BSSIDs to individual physical routers was not independently verified. Multiple advertised network names/radios must not be counted as independent routers.

Collected calibration samples

Anonymous student-network radio	Initial inside	Far-right back	Far-left back	Outside doorway, door closed
A, 2.4 GHz	-46	Not observed	-66	Not observed
B, 5 GHz	-52	-60	-60	-96
C, 2.4 GHz	-54	-63	-76	Not observed
D, 5 GHz	-57	-56	-54	-98
Total displayed BSSIDs	17	12	36	8

Values are dBm, read from the SPVS diagnostic UI. The user moved to each position, scanned and returned without rescanning; the result was preserved before any subsequent check. The 36-entry result was scrolled and captured completely. Positions and the closed door are user-reported, not independent location tracking. UI inspection times are not independent radio-acquisition timestamps. Absence from one scan does not prove an AP is physically absent.

Two consistently observed 5 GHz radio identifiers (B and D) were selected as **provisional references at -70 dBm each**. This is 10 dB below the weakest selected inside observation (-60). The existing rule is unchanged: at least half the configured references must meet their minima, so **one of these two is enough**. No stricter two-router co-presence or cryptographic proximity claim is made.

Offline checks using these measured scans and identical synthetic eligible GPS returned PRESENT for all three inside samples and PENDING_REVIEW with WIFI_GPS_DISAGREEMENT for the corridor sample. Those checks did not submit attendance. The threshold was fitted to these samples; fresh independent tests are required and no statistical accuracy rate is claimed.

Fresh attendance submissions under the earlier flow

Case	Server outcome	Evidence	Official record
Inside C003, 26 October 18:34:48 IST	PRESENT / ALL_REQUIRED_CHECKS_PASSED	GPS distance 13.5 m, uncertainty 20 m, age 0.6 s; liveness passed; scan available; Wi-Fi match ratio 0.5	PRESENT, source VERIFICATION
Hostel room, 26 October 23:57:53 IST	REJECTED / OUTSIDE_GEOFENCE	GPS distance 151.1 m, uncertainty about 26.4 m, age 0.7 s; liveness passed; scan available	None

The inside session was closed with its automatic record intact. A separate outside-test session avoided duplicate-attendance rejection. The user later clarified that the second attempt was made from the **hostel room**, not just outside C003. The test notes were corrected accordingly.

The server's geofence check returned before Wi-Fi matching in that hostel attempt. It is therefore evidence of a far-location GPS rejection, not proof that Wi-Fi excluded a person just outside the classroom. The earlier closed-door corridor diagnostic scan and the later hostel attendance submission are distinct cases.

Preserved state after the upstream update

The new database migration preserved C003, its two references, prior attempts, automatic record, and the separate hostel demo correction/dispute history. The old evidence remains unchanged. New attendance now requires mandatory QR/audio proof before face/GPS/Wi-Fi decisions; enrollment retains its own flow.

Pending physical work

Do not start another physical test until the user explicitly says they are in the classroom and ready. The next test plan must account for the new QR/audio flow, install the rebuilt native clients, use a fresh session and start faculty challenges. Record which gate determines any outside result; a QR/audio/GPS failure does not isolate the contribution of Wi-Fi. Include open-door/closed-door and adjacent-room conditions and new samples, not just the calibration readings above.

The immediate-doorway fresh Wi-Fi-specific attendance comparison was never completed under the older flow. Physical QR/audio interoperability, acoustic leakage/relay, and classroom automatic acceptance/rejection under policy v2 all remain unverified. See the physical plan.

Sanitized evidence: c003-validation.json. Raw identifiers, coordinates and UI snapshots remain in the ignored local work directory. No policy was weakened to force an attendance result.

SPVS local and GitHub synchronization - 27 October 2026

Publication: PR #7 from `spvs/classroom-sync-20261027` into main. The linked PR/Git refs are authoritative for current merge state. Integration checkpoint: `0fc235f916fb662db71c298091255ec10f0917f6`.

Incoming GitHub changes

Local main was clean at `3a92b73` and two commits behind. Upstream feature `9a3640e` and merged main `404eed8` (PR #6) add dynamic QR/audio protocol, database migration, dashboard controls and dependencies, native microphone modules, permissions, automated coverage and physical-test documentation. The local checkout was fast-forwarded; none of this upstream work was replaced.

Local system update

Dashboard dependencies were refreshed, a private database backup verified, and migration `9bd32a88f101` applied. Counts of existing accounts/devices/profiles, sessions/attempts/records/corrections/disputes were preserved. C003 retains its 50 m radius and two provisional AP references. Review policies remain unchanged. The API/dashboard were restarted and the new API route verified.

Android and both iOS native targets were rebuilt with SpvsAcoustic. Podfile.lock was updated, APK and signed-app verification passed, and generated microphone configuration was checked. The new APK replaces the old native host. No app was installed/launched on a phone and no physical trial or classroom challenge started.

Evidence and release refresh

- 146 backend, 64 mobile, 11 dashboard and 10 native Wi-Fi tests passed locally:

231 total, zero failures/skips.

- TypeScript, dashboard build, both JS exports, migration/drift and native builds

passed. Upstream cloud browser/container results remain separate evidence.

- The new C003 report records three inside

calibration points, the closed-door corridor scan, provisional -70 dBm references, an older-flow automatic inside PRESENT and the corrected hostel GPS rejection.

- The runtime report separates native compilation

from unperformed QR/audio hardware testing and describes the compatibility change.

- Current entry points, manuals, architecture/security/API/database guides and

delivery files are reconciled with mandatory policy v2.

- Source ZIP, PDFs, packaged Markdown and hash metadata are regenerated together.

The manifest records actual source content without embedding its own future commit.

What remains local or pending

Private backup/database data, credentials, signing material, raw router identifiers, precise coordinates, raw UI captures, templates, frames and audio remain excluded. Git synchronization does not deploy a cloud service or replicate local data.

The immediate-doorway Wi-Fi-specific attendance comparison was not completed. The later attempted outside submission actually came from the hostel and failed GPS before Wi-Fi evaluation. Earlier physical successes predate QR/audio.

All new physical tests wait for the user to say they are in the classroom and ready. Newly built clients must be installed and faculty QR/audio controls enabled for the next fresh session. Acoustic reliability/leakage/relay, actual new-flow classroom decisions, attestation, biometric calibration and production gates remain.

Fresh npm audit: 4 high mobile affected entries and zero dashboard findings. Backend vulnerability scanning is historical; no forced downgrade was performed.

After merge/fetch, local main and origin/main should be identical with a clean working tree. Verify with:

```
git status --short --branch
git rev-parse main origin/main
git rev-list --left-right --count main...origin/main
```

Expected main divergence: `0 0`. Older dated audit/sync records preserve their historical state and must not be read as today's pending-work inventory.

Local integration of GitHub QR/acoustic update

Date: **27 October 2026, Asia/Kolkata**. Incoming GitHub main: `404eed84cbb9f5f2eb0bc73e7b66910fb49476f9`, merged through PR #6. Feature implementation: `9a3640e2bc25f07da9dd450711c933f7d18624e7`. The prior local main (`3a92b73`) was clean and two commits behind, so it was fast-forwarded without conflicts or history rewriting. The incoming change covered 49 files, including the migration, protocol, UI, native modules and docs.

Updated runtime

- Installed the dashboard's locked dependencies, including QR rendering support.

- Created and checked a private PostgreSQL backup in the ignored work directory

before migrating. The backup contains private application data and is not an artifact.

- Upgraded the local database from `7a2d13ce4f90` to `9bd32a88f101`; Alembic model

drift check passed. Existing counts remained 4 users, 2 devices, 2 biometric profiles, 4 sessions, 4 attempts, 2 official records, 1 correction and 2 disputes.

- C003 coordinates/references and existing attendance outcomes were preserved.

iOS and unavailable-Wi-Fi policies remain review. No classroom challenge was automatically enabled and no new physical submission was made.

- Restarted the API and dashboard. API health/readiness passed, models loaded,

and the new `/api/v1/presence/evidence` route appeared in the running schema.

- Regenerated native projects with microphone permission/module declarations,

preserving local native settings. CocoaPods now includes SpvsAcoustic; the resolved Podfile.lock is retained in Git.

Local validation

Check	Local result
Backend	146 passed
Mobile	64 passed; TypeScript passed
Dashboard	11 passed; production build passed
Native Wi-Fi	10 passed
Total local tests	231 passed; no failures/skips in these suites
Android/iOS JS export	Passed; bundling is not hardware validation
Android arm64 APK	Native build passed; acoustic class present in DEX; signature verified
iOS simulator	Xcode BUILD SUCCEEDED; SpvsAcoustic Swift source compiled
iOS device app	Signed generic-device build passed; codesign verification passed
Native configuration	Microphone/module declarations verified
Browser E2E	Upstream cloud's 3 passing cases retained; not rerun against the local live demo database
Containers	Upstream build evidence retained; not rebuilt/deployed locally in this update

The cloud report's total is 234 because it includes three browser tests. The fresh local total is 231; do not add repeated runs as new independent cases. Upstream deprecation/SDK warnings remain. Actual local results and hashes are in results.json.

The rebuilt local Android APK is 78,138,028 bytes; SHA-256 `6cd0d397cccbbc42f6033c9a98eeab5a7c0d2f3f3d312b9dfdef72c607ab35c2`. The cloud-only APK had a different build hash and was not distributed in Git. The local replacement includes the required native microphone module.

Compatibility and policy transition

New attendance requires a rotating QR proof and a server-verified acoustic response before face/liveness, device, GPS and platform-presence checks can pass. Faculty must start challenges, display QR and play the queued student's signal. The capture is foreground/bounded and PCM stays transient on the phone.

QR rotates every five seconds with at most ten-second validity; audio redemption has a server-clock five-second deadline. Individual audio playback reservations serialize the channel to at most 12 per minute. Recordings/live relay and sound leakage remain possible; this is not distance-bounding or certified proximity.

The migration retires old iOS gps_only policy into dynamic if present, but the local review policy was unchanged. Dynamic iOS approval is a separate audited administrator choice and was not enabled here. Provider attestation remains unconfigured; QR/audio proof is not independent platform/sensor attestation.

Previously installed phone builds lack SpvsAcoustic and must be replaced before testing the new flow. **No rebuilt client was installed or launched on a phone during this update.** The signed iPhone app remains in ignored local build output; no signing keys/profile or production IPA is published. Native compilation is not microphone permission,

speaker/microphone or acoustic-channel validation.

Security, privacy and remaining work

Fresh npm audit: 4 high affected mobile entries and zero dashboard findings. No forced dependency downgrade was applied. Backend vulnerability scanning is historical, not a current clearance. Genuine provider attestation, calibrated biometrics, load/throughput, retention/backups, production TLS/signing and institutional policy remain release gates.

The C003 report records the earlier inside automatic pass and the corrected hostel GPS rejection. They predate policy v2 and do not demonstrate physical QR/audio success. The old immediate-doorway Wi-Fi-specific attendance test was not completed.

All physical testing waits for explicit user readiness in the classroom. Secrets, database backup/data, raw AP identifiers, exact locations, private keys, biometric templates, raw frames/audio and raw UI snapshots stay excluded from Git.

Dynamic QR + acoustic attendance (policy v2)

This feature adds two mandatory presence factors to the existing identity, randomized liveness, registered-device signature, GPS, review, dispute and audit workflows. It reduces reliance on iOS Wi-Fi capabilities. It does **not** prove proximity: QR screenshots, recordings and live relays can carry valid challenges off-site.

Protocol and authority

1. The student selects a session and obtains `/challenges` with purpose attendance.

The existing challenge UUID is the coordinated attempt ID throughout QR, audio, face capture and the final signed `/attendance/verify` request. It lasts 120 seconds. One unused, unexpired challenge per student/session is reused on interrupted requests. Issuance is bounded by the configured `max_attempts`, including abandoned attempts.

2. Authorized course faculty starts challenges. `/presence/display` produces a

cryptographically random QR token on each five-second server clock boundary. The token has no student data; its HMAC binds its random ID/nonce to the persisted session, room and display epoch. Each challenge is valid for at most ten seconds; equality with expiry is rejected. The dashboard polls and hides stale codes.

3. The phone signs a QR redemption using its existing registered key. The server

authenticates the student, device, enrollment, active session and token. A shared classroom QR may be redeemed by many students, but only once per student across attempts. Retrying the same bound attempt returns the previous acknowledgement.

4. After permission, the phone starts foreground microphone capture **before** sending

signed `ready`. Faculty sees only waiting attempt labels and presses Play. The server issues one fresh answer for that attempt; only authorized faculty receives it over the API. Student-accessible responses never reveal it.

5. The laptop plays a 2.7-second signal. The phone decodes locally, ignores other

attempt tags, and sends the decoded value in a device-signed message immediately. No caller-supplied pass flag is accepted. Wrong or expired answers burn the challenge. A correct response is persisted, and exact response retries return the same result without consuming another challenge.

6. Face/liveness, fresh GPS, available Wi-Fi and device proof use the existing flow.

The final transaction checks all mandatory factors before creating a record. The immutable attempt retains reasons and the policy version; ordinary faculty correction/dispute workflows remain available and never rewrite the original decision.

Timing and concurrency

The acoustic five-second deadline starts when the server issues the faculty play response, **before network transit or browser scheduling**. It includes faculty response latency, playback startup, 2.7 s transmission, phone processing/signing and return transit. Approximately 2.3 s remain for those other operations. The dashboard refuses to begin playback if its issuance round trip exceeds 1.2 s. There is no client-clock grace period or deadline extension; slow networks require a fresh attempt or review. Phone capture waits at most 28 s for faculty, native capture at most 30 s; waiting does not start the acoustic deadline. HTTP requests time out after 20 s.

Database transactions lock student → device → liveness challenge → session → proof. Faculty operations lock session then proof and never acquire student/challenge locks. Final verification and corrections share the session lock. QR issuance is serialized by that same lock, so multiple faculty tabs share one QR for the current interval. Unique `(qr_id, student_id)` and existing attendance/idempotency constraints provide additional replay/duplicate protection. The server

clock is read after lock acquisition. Rate buckets are shared PostgreSQL records, not per-process counters.

Stopping/restarting challenges advances a display epoch and invalidates all previously bound proofs for unfinished attempts. Closing/expiring a session prevents acceptance at every stage. It does not erase historical records. Audio is serialized per session with a five-second channel reservation: maximum 12 individual playbacks/minute. This first implementation intentionally exposes that capacity limit; large classes need staggered students, longer sessions or a separately designed batch protocol. A waiting student should only tap Listen when faculty can play within 28 seconds.

Audible codec and phone capture

`mobile/src/acousticCodec.ts` is used by both the dashboard and student app:

- 16-FSK carriers: 1,200 Hz + 160 Hz × symbol (through 3,600 Hz).

- 75 ms symbols, an eight-symbol synchronization preamble, then a 12-byte message

(32-bit public attempt routing tag + 64 unpredictable bits) and CRC16-CCITT.

- 36 symbols total: 2.7 seconds, with short amplitude ramps at symbol edges.

- Goertzel energy comparison on symbol centers; timing-offset search, carrier

dominance threshold and checksum reject ambiguous/corrupt captures. Synthetic tests include unaligned capture, attenuation, clipping, additive noise and echo.

- CRC is error detection, **not authentication**. Server randomness, signature

authentication, attempt binding, one guess and expiry provide protocol checks. No FEC is claimed; damaged frames require retry/review. Real microphone clock drift, reverberation and AGC performance require the physical-device guide.

- Kotlin uses mono 16 kHz PCM with `VOICE_RECOGNITION`; Swift uses AVAudioEngine

measurement mode, reporting the actual sample rate after integer decimation. Native errors and missing permissions fail closed. Bluetooth routing, phone calls, backgrounding, aggressive audio processing or inaccessible audio may prevent capture.

Use local laptop speakers at a comfortable audible volume, never ultrasound. Do not interpret successful decoding in a corridor or adjacent room as a software defect that can be solved solely by higher volume or a shorter token lifetime.

Policy migration and compatibility

Apply Alembic revision `9bd32a88f101` before deploying the API/clients. It adds `presence_qr`, `presence_proofs`, and session enable/epoch fields. Historical attempts, records, biometrics, devices and corrections are unchanged. Displays start **stopped**. Old clients cannot auto-approve without QR/audio: they return PENDING_REVIEW with specific reasons. Enrollment and legacy registered HMAC-device compatibility remain. New device registrations still require the existing native-key flow.

Both Android and iOS require QR/audio unconditionally; there is no bypass switch. For iOS, the new `dynamic` policy permits automatic attendance only after QR/audio and all existing mandatory checks. Existing `review`/`retry` policies stay stricter. The migration converts existing iOS `gps_only` to `dynamic`; new policies default `dynamic`. A later API write of legacy `gps_only` still cannot bypass QR/audio and retains its existing App Attest requirement. The dashboard offers `dynamic`, review or retry. Required provider attestation remains a separate gate. Android Wi-Fi scan evaluation and its existing `wifi_unavailable` policy remain: review/reject or optional Wi-Fi with QR/audio/GPS. Optional iOS current-network identity never becomes a nearby Wi-Fi scan or fabricated RSSI.

Deploy backend, dashboard and rebuilt development apps together. Native microphone code requires a new Android/iOS build; Expo Go and existing APK/IPA artifacts do not include it. The tracked historical Podfile.lock is preserved; `pod`

`install` on the Mac must regenerate it for the new local module. No Xcode compilation is implied by Expo prebuild or an iOS JavaScript export.

API additions

All paths below are under `/api/v1`, authenticated and course-scoped.

Endpoint	Actor / purpose
POST `/sessions/{id}/presence/control`	Faculty/admin `{enabled}`; audited start/stop
GET `/sessions/{id}/presence/display`	Faculty/admin QR, authoritative timing, listening queue
POST `/sessions/{id}/presence/play/{attempt_id}`	Faculty/admin one-use audio payload and deadline
POST `/presence/evidence`	Student device-signed `qr`, `ready`, `audio`, or `failure` message

Evidence uses the existing SignedEvidence envelope: exact `payload` JSON string, `signature` over `idempotency_key + '.' + payload`, and a random idempotency key. Presence payload always includes `device_id`, `challenge_id`, `action`; QR adds `token`, audio adds `response`, failure adds an allowlisted reason. A failure creates PENDING_REVIEW, with identity marked NOT_COMPLETED, and never an attendance record. Explicit faculty corrections continue to require an audit reason. HTTP 409 means invalid/expired/used/mismatched factor. HTTP 429 is a bounded limit. Lost acknowledgements can retry exact evidence; expired challenges cannot be revived.

Privacy and remaining security boundaries

Microphone samples live in a bounded JS/native memory buffer only. No audio file, recording upload or microphone audit payload is created. Stop/unmount/background clears buffers and ends capture. Audits retain attempt IDs, timestamps and reason codes, not nonces, audio, biometric templates or face frames. QR tokens and audio hashes live in protocol tables rather than audit details. The existing encrypted biometric profile storage is unchanged. Operators should delete expired unused protocol records under their institutional retention policy; retain referenced records needed for disputes. No new automated deletion schedule is enabled.

A malicious modified client can submit a relayed answer; signed data proves the registered key signed it, not that a real microphone captured it. A trusted faculty account can also leak answers. The 32-bit routing tag is not a secret and is only a receiver filter; the complete random response is checked. Live relay within five seconds remains possible. GPS uncertainty, biometric spoof resistance, provider attestation and device integrity retain their previously documented limitations. Physical validation and institutional approval remain production release gates.

Physical-device validation: QR + audible acoustic attendance

Status: not performed for this feature. Earlier SPVS hardware reports concern the older flow. Automated PCM tests, JS exports and Android compilation do not establish that a laptop-to-phone acoustic channel works in a real classroom.

The 27 October local integration has now applied the migration and rebuilt Android, iOS simulator and signed-device apps. Those builds were not installed on phones. Start physical tests only after the user confirms classroom readiness; preserve existing credentials with an in-place development-client update rather than uninstalling a registered app.

Setup on MacBook Air M4

Use a native ARM64 terminal, current Xcode/command-line tools, Docker Desktop, Node 24 LTS, Python 3.12 + uv, Android Studio/SDK, JDK 17, and the repository's Bundler/CocoaPods setup. Preserve your existing uncommitted work before switching. From your existing repository checkout:

```
git fetch origin
git switch main
git pull --ff-only
make setup
make db
make migrate
make seed
make build
```

The commands above cover a fresh setup. In the already configured local Mac environment, the database migration is complete; do not reinitialize its database or reseed merely to resume testing. Use the existing API/dashboard/Metro tasks and the verified rebuilt clients. Preserve the C003 references and prior records.

Keep development services in separate terminals:

```
make api
make dashboard
```

Run automated checks (the test database name must end `_test`):

```
docker compose exec -T db createdb -U spvs spvs_test  # once; skip if it exists
make test
(cd mobile && npm run export)
(cd dashboard && npx playwright install chromium && npx playwright test)
```

For a LAN HTTP development demo only, replace the address below with your Mac's reachable Wi-Fi address. The phone and Mac must reach each other; campus client isolation can block this. Use trusted HTTPS for release builds.

```
export EXPO_PUBLIC_API_URL="http://$(ipconfig getifaddr en0):8000/api/v1"
export SPVS_DEV_HTTP=1
bash scripts/build_mobile_macos.sh ios-simulator
bash scripts/build_mobile_macos.sh ios-device
# Xcode opens: choose your Apple team and connected iPhone 16 Pro, then Run.
# Separate terminal with the same environment:
(cd mobile && npm start)
```

For Motorola Edge 50 Neo, enable USB debugging and authorize the Mac:

```
export EXPO_PUBLIC_API_URL="http://$(ipconfig getifaddr en0):8000/api/v1"
export SPVS_DEV_HTTP=1
export SPVS_ANDROID_ARCHITECTURES=arm64-v8a
bash scripts/build_mobile_macos.sh android
adb devices -l
adb install -r mobile/android/app/build/outputs/apk/debug/app-debug.apk
```

```
(cd mobile && npm start)
```

These commands must create **new** native builds containing `SpvsAcoustic`. The historical release APK does not include this module. Do not commit local `.env`, private keys, profiles, databases, recordings or biometric material. If provisioning changes, follow the existing iOS guide. Connected-Wi-Fi entitlement and App Attest remain separate opt-ins; nearby Wi-Fi scanning is never available on iOS.

Baseline run

1. Use consented test accounts, one per device. Confirm enrollment and native key

registration; configure the actual classroom geofence. Keep mandatory checks on. Set iOS policy to `dynamic` through the audited admin policy editor. Keep Android Wi-Fi requirements explicit; collect real references if Wi-Fi is required.

2. Open the faculty session page. Start challenges; verify a new QR every five

seconds and automatic hiding at expiry or loss of connection. Confirm browser speakers work at comfortable volume before students start.

3. Select session on the phone, scan QR, tap Listen, and read the eight-character

attempt label to faculty. Play that matching queue entry. Stay foreground; disconnect headphones/Bluetooth routing. Only then complete face/liveness/GPS.

4. Confirm server PRESENT only after every mandatory gate passes. Confirm one

official record and one final attempt even when retrying a lost final response.

5. Deny microphone/camera, then use Finish attempt & request faculty review.

Confirm PENDING_REVIEW, identity NOT_COMPLETED, and no automatic official record. Exercise dispute, reasoned faculty correction and resolution; retain original attempt and correction provenance. Do not call a manual correction an automatic pass.

Measurement matrix

Run every row on **iPhone 16 Pro** and **Motorola Edge 50 Neo**. Record OS/build, phone orientation/case, laptop/speaker model, browser, volume, distance, ambient condition, network RTT, attempt ID, decoder outcome, server reason, and latency. Collect at least 20 trials per ordinary seating/ambient combination; retain failures. Use anonymous location labels instead of exact personal coordinates or identifiers.

Condition	Procedure and expected interpretation
Front/middle/back, corners, obstructed seats	Measure decode success and latency; failed capture never auto-passes
Quiet and loud classroom	Speech, fans and ordinary classroom noise; vary safe volume, never discomfort
Doorway, corridor, adjacent room (door open/closed)	Measure sound leakage; successful decode outside is possible and is a limitation
Poor QR lighting, glare, distance	Exercise torch, rescan, rotation overlap and expired screenshot rejection
Recorded audio	Old recording must fail in a fresh attempt; current recording replay within deadline may succeed and must be reported

Condition	Procedure and expected interpretation
Live QR sharing	Share a current screenshot; QR alone must never suffice; sharing within ten seconds may redeem QR
Real-time QR + audio relay	Consenting tester relays both; record success/latency without claiming relay resistance
Simultaneous listeners	Queue different attempt tags; each phone ignores the other's frame; two faculty tabs cannot overlap issuance
Camera/mic denied or revoked	Clear guidance, no automatic pass, bounded retry/review; restore permissions in Settings
Call, lock screen, background, Bluetooth route	Capture stops or fails; no fabricated success, no lingering recording indicator
Network disabled/delayed/dropped acknowledgement	Exact retries recover accepted evidence; late answers fail; final submission is idempotent
Session closed or display stopped/restarted	Pending and prior verified proofs cannot produce new automatic attendance
GPS inaccurate/outside/mock, wrong face, failed liveness, revoked device	Valid QR/audio cannot override the failing mandatory gate
VoiceOver/TalkBack and audio inaccessible	Controls are identifiable; obtain faculty review without requiring inaccessible audio
Restart app midway / exhaust attempt budget	No automatic attendance; bounded attempts and review remain enforceable

Do not upload recordings by default. If an approved acoustic study genuinely needs recordings, obtain specific consent, use separate secure storage and a deletion schedule; the application itself deliberately has no raw-audio upload route.

Release gate and result sheet

Define institutional thresholds **before** collecting results (in-room completion rate, latency distribution, acceptable leakage/relay risk, review burden). Do not select thresholds just to fit observed results. Report counts, median/p95 latency, all reason codes, failures and network conditions separately for both phones.

For each trial: `device | OS | build SHA | scenario | seat label | attempt ID | QR result | audio result | end-to-end latency | final state | reason | review source`. Mark native Xcode compilation, installation, permission behavior, real audio interoperability and end-to-end hardware attendance separately. Any unperformed item stays **NOT TESTED**. Physical leakage or real-time relay success must remain in the final limitations report, even when every automated test passes.

References and provenance

The following are implementation references, not claims of deployment or experimentally measured accuracy. Consult the linked versioned source and official documentation when reproducing the project.

1. FastAPI documentation: https://fastapi.tiangolo.com/ — dependency injection, validation, security and API documentation.

2. SQLAlchemy 2.0 documentation: https://docs.sqlalchemy.org/en/20/ — ORM mappings, sessions and transactions.

3. PostgreSQL 16 documentation: https://www.postgresql.org/docs/16/ — constraints, indexes and row-level locking.

4. Alembic documentation: https://alembic.sqlalchemy.org/ — migration generation and application.

5. OpenCV Zoo, pinned commit: https://github.com/opencv/opencv_zoo/tree/48b10fa505aeb51fbd127fc60a5c660040e823b7 — official YuNet/SFace assets, Git LFS object hashes and licenses.

6. OpenCV DNN face tutorial: https://docs.opencv.org/4.x/d0/dd4/tutorial_dnn_face.html — detection, aligned SFace features and matching metrics. Its example threshold is not a local SPVS calibration result.

7. MediaPipe Face Landmarker guide: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker — Tasks API and official model bundle.

8. MediaPipe source/license: https://github.com/google-ai-edge/mediapipe — Apache 2.0 code and model-related resources.

9. Argon2-cffi documentation: https://argon2-cffi.readthedocs.io/ — maintained password hashing interface.

10. Cryptography Fernet documentation: https://cryptography.io/en/latest/fernet/ — authenticated symmetric encryption and key handling.

11. Expo documentation: https://docs.expo.dev/ — development builds, Camera, Location, SecureStore and local modules.

12. Android Wi-Fi scanning overview: https://developer.android.com/develop/connectivity/wifi/wifi-scan — scan restrictions, permissions and throttling.

13. Android Play Integrity overview: https://developer.android.com/google/play/integrity/overview — external provider integration requirements.

14. Apple App Attest: https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity — attestation boundary not configured in this distribution.

15. React, Vite and TanStack Query: https://react.dev/ ; https://vite.dev/ ; https://tanstack.com/query/latest — client architecture.

16. Playwright and pytest: https://playwright.dev/ ; https://docs.pytest.org/ — browser and backend testing.

17. scikit-image astronaut sample, version 0.24.0: https://github.com/scikit-image/scikit-image/blob/v0.24.0/skimage/data/astronaut.png — public NASA astronaut portrait used for model smoke tests.

18. matplotlib Grace Hopper sample, version 3.9.2: https://github.com/matplotlib/matplotlib/blob/v3.9.2/lib/matplotlib/mpl-data/sample_data/grace_hopper.jpg — public reference portrait used for a different-person smoke test.

19. npm advisory GHSA-vfj7-8cjw-p6xm: https://github.com/advisories/GHSA-vfj7-8cjw-p6xm — unresolved `braces` issue recorded by audit.

20. npm advisory GHSA-86w9-cpqp-85rv: https://github.com/advisories/GHSA-86w9-cpqp-85rv — unresolved `node-forge` issue recorded by audit.

License texts for the downloaded face detector and recognizer are included in `LICENSE-YUNET.txt` and `LICENSE-SFACE.txt`. The project source inherits the repository's MIT license. Third-party dependency licenses remain their respective authors' licenses. Reference portraits are downloaded only for local tests and are excluded from submission archives.